Unit 5

Virtualization and Containerization Technology

(8 Hours)

. 5.1 Introduction to virtualization

. 5.2 Types of virtualization:

. Server virtualization

. Storage virtualization

. Network virtualization

. 5.3 Virtualization architecture and implementation

. 5.4 Hypervisors and types

. 5.5 Virtual machine lifecycle

. 5.6 Introduction to containerization

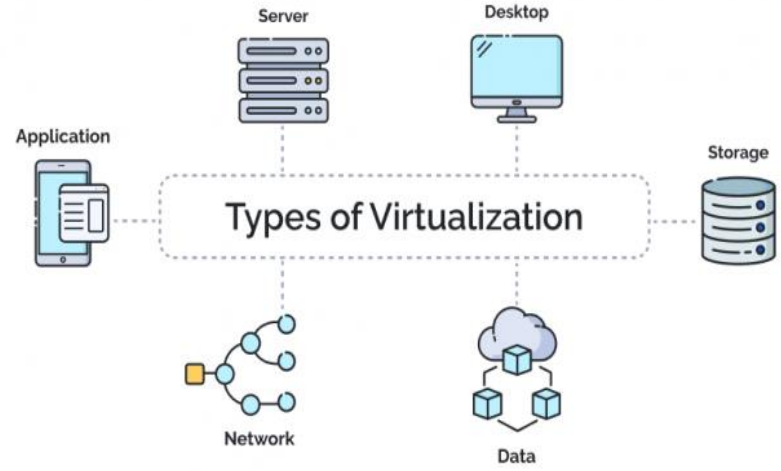
. 5.7 Virtual Machine vs Container

. 5.8 Introduction to serverless computing

. 5.9 Event-driven functions

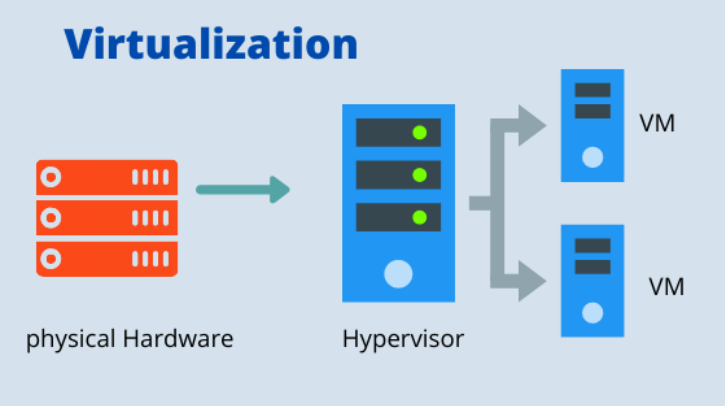
. 5.10 Load balancing concepts

. 5.11 Serverless vs containerized workloads



Sanjeev Thapa, Er. DevOps, SRE, CKA, RHCSA, RHCE, RHCSA-Openstack, MTCNA, MTCTCE, UBSRS, HEv6, Research Evangelist

. 5.1 Introduction to virtualization



Meaning of Virtualization

Virtualization is a technology that allows one physical computer/server to act like multiple separate computers.

Each virtual computer is called a **Virtual Machine**, commonly known as **VM**.

A virtual machine has its own:

. *Operating system*
. *CPU allocation*
. *RAM allocation*
. *Storage*
. *Network interface*
. *Applications*

But physically, all VMs run on the same hardware.

Simple Example

Suppose one physical server has:

Resource	Capacity
CPU	16 cores
RAM	64 GB
Storage	2 TB
Network	1 Gbps

Using **virtualization**, we can create multiple virtual machines:

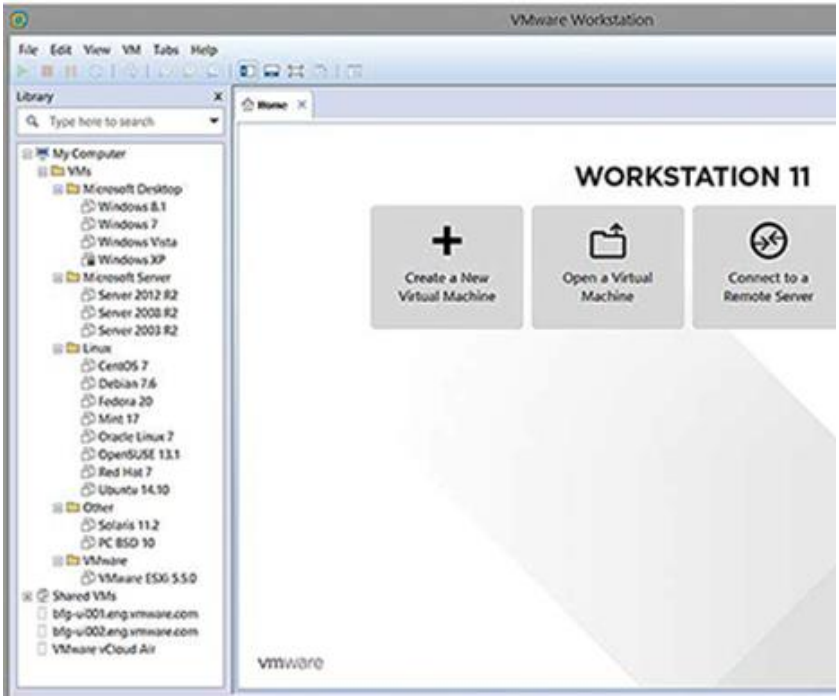
VM Name	CPU	RAM	Storage	Purpose
VM-1	4 cores	16 GB	500 GB	Web Server
VM-2	4 cores	16 GB	500 GB	Database Server
VM-3	2 cores	8 GB	250 GB	Testing Server
VM-4	2 cores	8 GB	250 GB	File Server

So, instead of buying 4 separate physical servers, we use one powerful server and divide it virtually.

Real-Life Analogy

Virtualization is like dividing one large building into many separate apartments.

> *The building is the physical server.*
> *Each apartment is a virtual machine.*
> *Each apartment has its own room, door, electricity, and users, but all are inside the same physical building.*



Why Virtualization is Needed

Before virtualization, many organizations used one server for one application.

Example:

. *One physical server for website*
. *One physical server for database*
. *One physical server for email*
. *One physical server for file sharing*

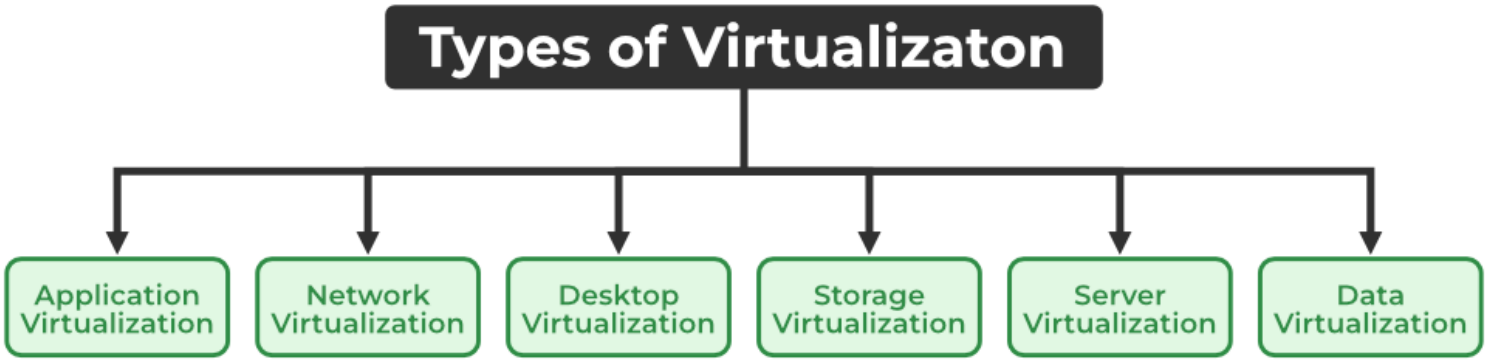
This caused:

. *High hardware cost*
. *Low resource utilization*
. *More electricity use*
. *More cooling requirement*
. *Difficult maintenance*

With virtualization:

. *One server can run multiple systems*
. *Hardware is used efficiently*
. *Cost is reduced*
. *Backup and recovery become easier*

. 5.2 Types of virtualization:



. Server virtualization *. Storage virtualization* *. Network virtualization*

There are different types of virtualization.

Main types:

. *Server virtualization*
. *Storage virtualization*
. *Network virtualization*
. *Desktop virtualization*
. *Application virtualization*

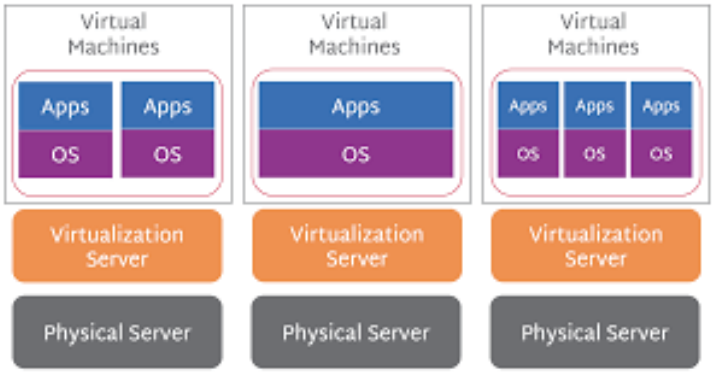
According to syllabus, we focus mainly on server, storage, and network virtualization.

A. Server Virtualization

Meaning

Server virtualization means dividing one physical server into multiple virtual servers.

Each virtual server runs independently.



Example

A college has one physical server:

Physical Server Resource	Capacity
CPU	24 cores
RAM	128 GB
Storage	4 TB

It can be divided into:

Virtual Server	CPU	RAM	Storage	Use
VM-Web	4 cores	16 GB	500 GB	College Website
VM-LMS	6 cores	32 GB	1 TB	Moodle / LMS
VM-DB	8 cores	48 GB	1 TB	Database
VM-Backup	4 cores	16 GB	1 TB	Backup Server

Benefits of Server Virtualization

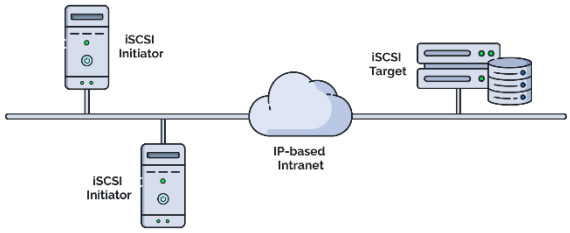
. *Reduces hardware cost*
. *Improves server utilization*
. *Easy backup and migration*
. *Easy testing and development*
. *Better disaster recovery*

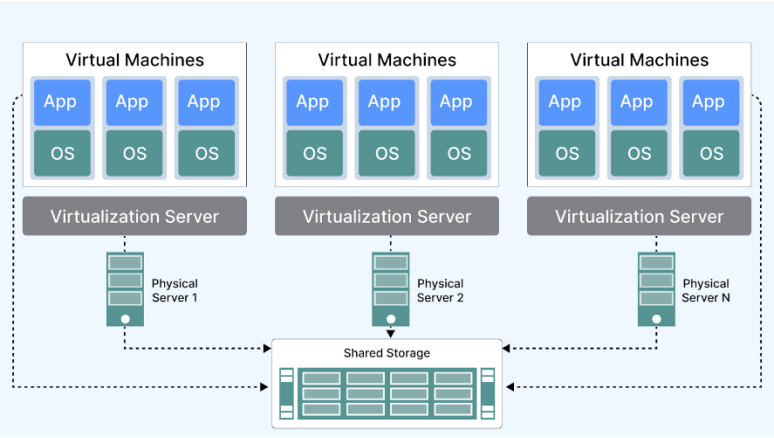
B. Storage Virtualization

Meaning

Storage virtualization combines multiple physical storage devices into one logical storage system.

Users see one storage system, but behind it, data may be stored across many disks or storage servers.





Example

Suppose a company has three storage devices:

Storage Device	Capacity
Disk Array 1	2 TB
Disk Array 2	3 TB
Disk Array 3	5 TB

Using storage virtualization, these are combined into one logical storage pool:

Total Virtual Storage = 2 TB + 3 TB + 5 TB = 10 TB

Users only see:

Company Storage Pool = 10 TB

Data-Based Example

A hospital stores the following data:

Data Type	Storage Required
Patient records	1 TB
Lab reports	2 TB
X-ray and scan images	4 TB
Backup data	3 TB

Total storage needed:

1 TB + 2 TB + 4 TB + 3 TB = 10 TB

Instead of managing separate disks manually, storage virtualization provides one central 10 TB storage pool.

Benefits of Storage Virtualization

. *Centralized storage management*
. *Better storage utilization*
. *Easy backup*

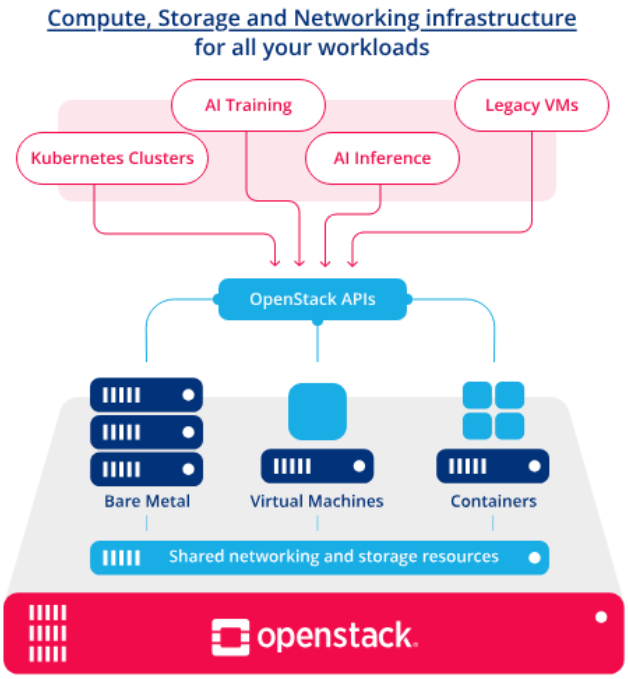
. *Easy expansion*
. *Improved data availability*

C. Network Virtualization

Meaning

Network virtualization means creating virtual networks using software instead of depending only on physical network devices.

It allows multiple logical networks to run on the same physical network infrastructure.



Example

A college has one physical network but wants to separate departments:

. *Admin Network*
. *Teacher Network*
. *Student Network*
. *Guest Wi-Fi Network*
. *Lab Network*

Using network virtualization, we can create VLANs or virtual networks.

VLAN-Based Example

Department	VLAN ID	IP Network
Admin	10	192.168.10.0/24
Teachers	20	192.168.20.0/24
Students	130	192.168.30.0/24
Guest Wi-Fi	40	192.168.40.0/24
Lab	50	192.168.50.0/24

Although all departments may use the same physical switch, their traffic is separated logically.

Benefits of Network Virtualization

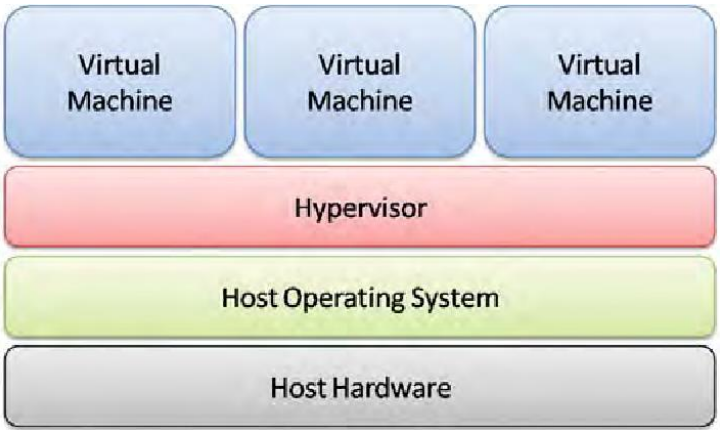
. *Better security*
. *Logical separation of users*
. *Easier network management*
. *Supports cloud networking*
. *Reduces dependency on physical devices*

. 5.3 Virtualization architecture and implementation

Basic Virtualization Architecture

```
+-----------------------------------------------+
|              Virtual Machine 1                |
|      OS + Application + Virtual Hardware      |
+-----------------------------------------------+
|              Virtual Machine 2                |
|      OS + Application + Virtual Hardware      |
+-----------------------------------------------+
|              Virtual Machine 3                |
|      OS + Application + Virtual Hardware      |
+-----------------------------------------------+
|                 Hypervisor                    |
+-----------------------------------------------+
|        Physical Hardware / Host Machine       |
|     CPU + RAM + Storage + Network Interface   |
+-----------------------------------------------+
```

Main Components of Virtualization Architecture



1. Host Machine

The physical computer or server where virtualization is installed.

Example:
A Dell, HP, Lenovo, or custom-built server with CPU, RAM, storage, and network card.

---

2. Hypervisor

The software layer that creates and manages virtual machines.

Examples:

. *VMware ESXi*
. *Microsoft Hyper-V*
. *KVM*
. *VirtualBox*
. *Proxmox VE*

---

3. Guest Machine

The virtual machine running inside the host.

Each guest machine may have its own operating system.

Example:

. *Ubuntu VM*
. *Windows Server VM*
. *Kali Linux VM*
. *CentOS VM*

---

4. Virtual Hardware

Virtual hardware is the simulated hardware given to each VM.

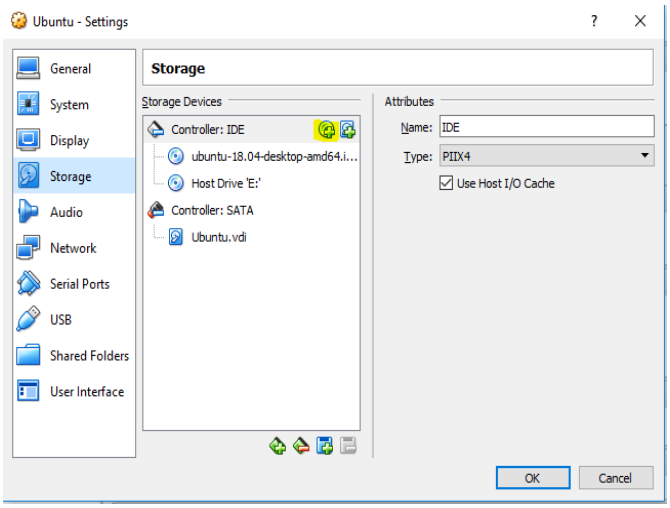
Examples:

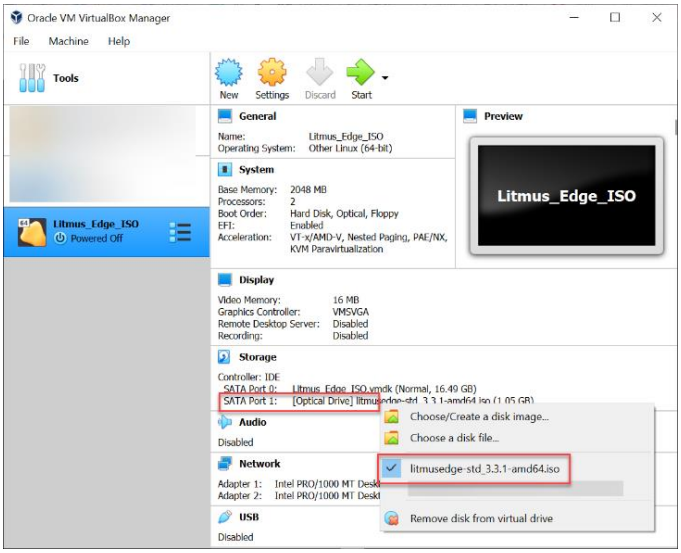
. *Virtual CPU*
. *Virtual RAM*
. *Virtual Hard Disk*
. *Virtual Network Adapter*
. *Virtual Display*

Implementation Example

Assume we have one server:

Physical Resource	Capacity
CPU	32 cores
RAM	128 GB
Storage	4 TB SSD
Network	10 Gbps





We want to create 5 VMs:

VM	CPU	RAM	Storage	OS	Purpose
VM1	4 cores	16 GB	500 GB	Ubuntu	Web Server
VM2	8 cores	32 GB	1 TB	Ubuntu	Database
VM3	4 cores	16 GB	500 GB	Windows Server	File Server
VM4	2 cores	8 GB	250 GB	Ubuntu	Testing
VM5	4 cores	16 GB	500 GB	Linux	Backup

Total allocated:

CPU = 4 + 8 + 4 + 2 + 4 = 22 cores
RAM = 16 + 32 + 16 + 8 + 16 = 88 GB
Storage = 500 + 1000 + 500 + 250 + 500 = 2750 GB

Remaining:

CPU = 32 - 22 = 10 cores
RAM = 128 - 88 = 40 GB
Storage = 4000 - 2750 = 1250 GB

This remaining resource can be used for future VMs.

. 5.4 Hypervisors and types

Meaning of Hypervisor

A hypervisor is software that creates, runs, and manages virtual machines.

It controls how physical resources are shared among VMs.

Types of Hypervisors

There are two main types:

. *Type 1 Hypervisor*
. *Type 2 Hypervisor*

A. Type 1 Hypervisor

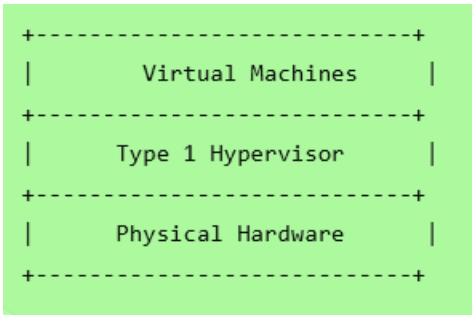
Meaning

A Type 1 hypervisor runs directly on physical hardware.

It does not need a host operating system.

It is also called a **bare-metal hypervisor**.

Architecture



Examples

. *VMware ESXi*
. *Microsoft Hyper-V Server*
. *KVM*
. *Xen*
. *Proxmox VE*

Use Case

Used in:

. *Data centers*
. *Cloud platforms*
. *Enterprise servers*
. *Production environments*

Advantages

. *High performance*
. *Better security*
. *Better resource control*
. *Suitable for production*

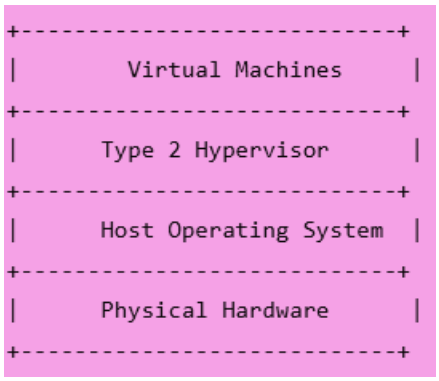
B. Type 2 Hypervisor

Meaning

A Type 2 hypervisor runs on top of an existing operating system.

It is also called a **hosted hypervisor**.

Architecture



Examples

. *Oracle VirtualBox*
. *VMware Workstation*
. *VMware Fusion*
. *Parallels Desktop*

Use Case

Used in:

. *Learning*
. *Testing*
. *Development*
. *Running Linux inside Windows*
. *Running Windows inside Mac*

Advantages

. *Easy to install*
. *Good for students*
. *Good for labs*
. *No dedicated server required*

Type 1 vs Type 2 Hypervisor

Feature	Type 1 Hypervisor	Type 2 Hypervisor
Runs on	Direct hardware	Host operating system
Performance	High	Medium
Security	Better	Lower than Type 1
Use	Production, data center	Learning, testing
Examples	ESXi, Hyper-V, KVM	VirtualBox, VMware Workstation
Cost	May need dedicated server	Can run on laptop

. 5.5 Virtual machine lifecycle

Meaning

Virtual machine lifecycle means all stages from creating a VM to deleting it.

VM Lifecycle Stages

1. Planning
2. Creation
3. Configuration
4. Installation
5. Running
6. Monitoring
7. Snapshot / Backup
8. Migration
9. Scaling
10. Shutdown / Suspend
11. Deletion

VM Lifecycle Diagram

Plan VM Requirement
↓
Create VM
↓
Allocate CPU, RAM, Storage
↓
Install Operating System
↓
Install Applications
↓
Run VM
↓
Monitor Performance
↓
Backup / Snapshot
↓
Update / Migrate / Scale
↓
Shutdown or Delete VM

Example: VM Lifecycle in College Lab

Step 1: Planning

Requirement:

Need one Ubuntu VM for cloud computing practical.

Step 2: Create VM

Create a VM in VirtualBox.

Step 3: Allocate Resources

Resource	Value
CPU	2 cores
RAM	4 GB
Storage	40 GB
OS	Ubuntu Linux

Step 4: Install OS

Install Ubuntu ISO file.

Step 5: Install Software

Install:

```
sudo apt update
sudo apt install nginx docker.io -y
```

Step 6: Run Services

Start web server:

```
sudo systemctl start nginx
```

Step 7: Monitor

Check CPU and memory:

```
top
free -h
df -h
```

Step 8: Snapshot

Take snapshot before risky changes.

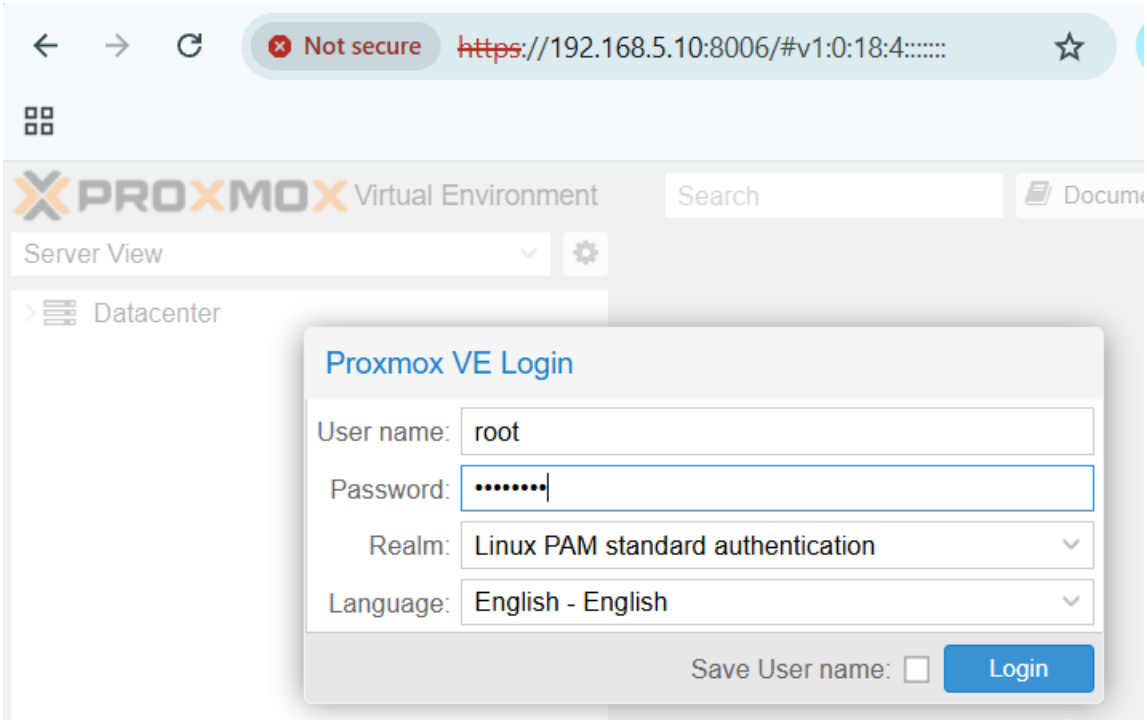
Step 9: Shutdown

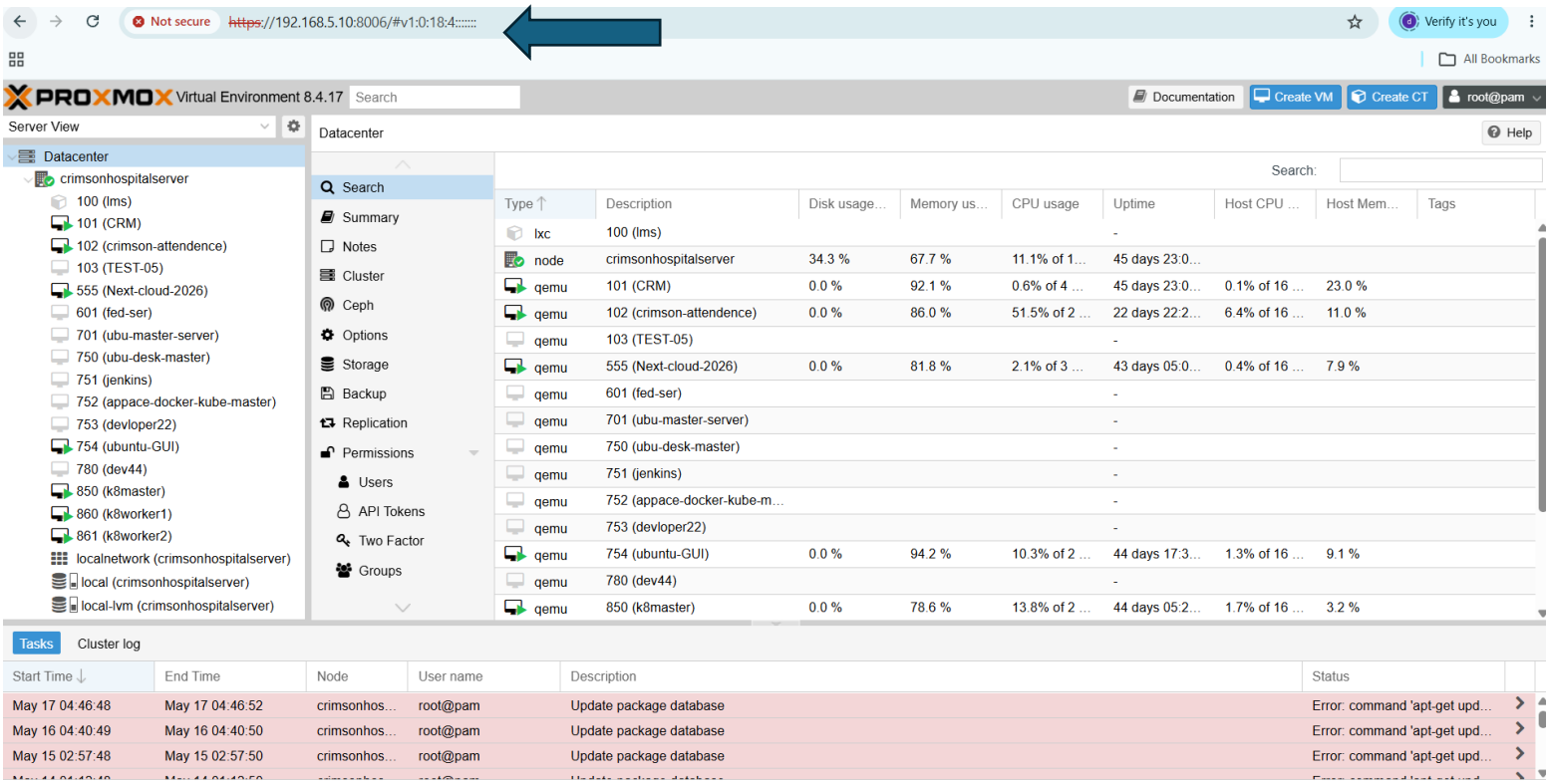
```
sudo shutdown now
```

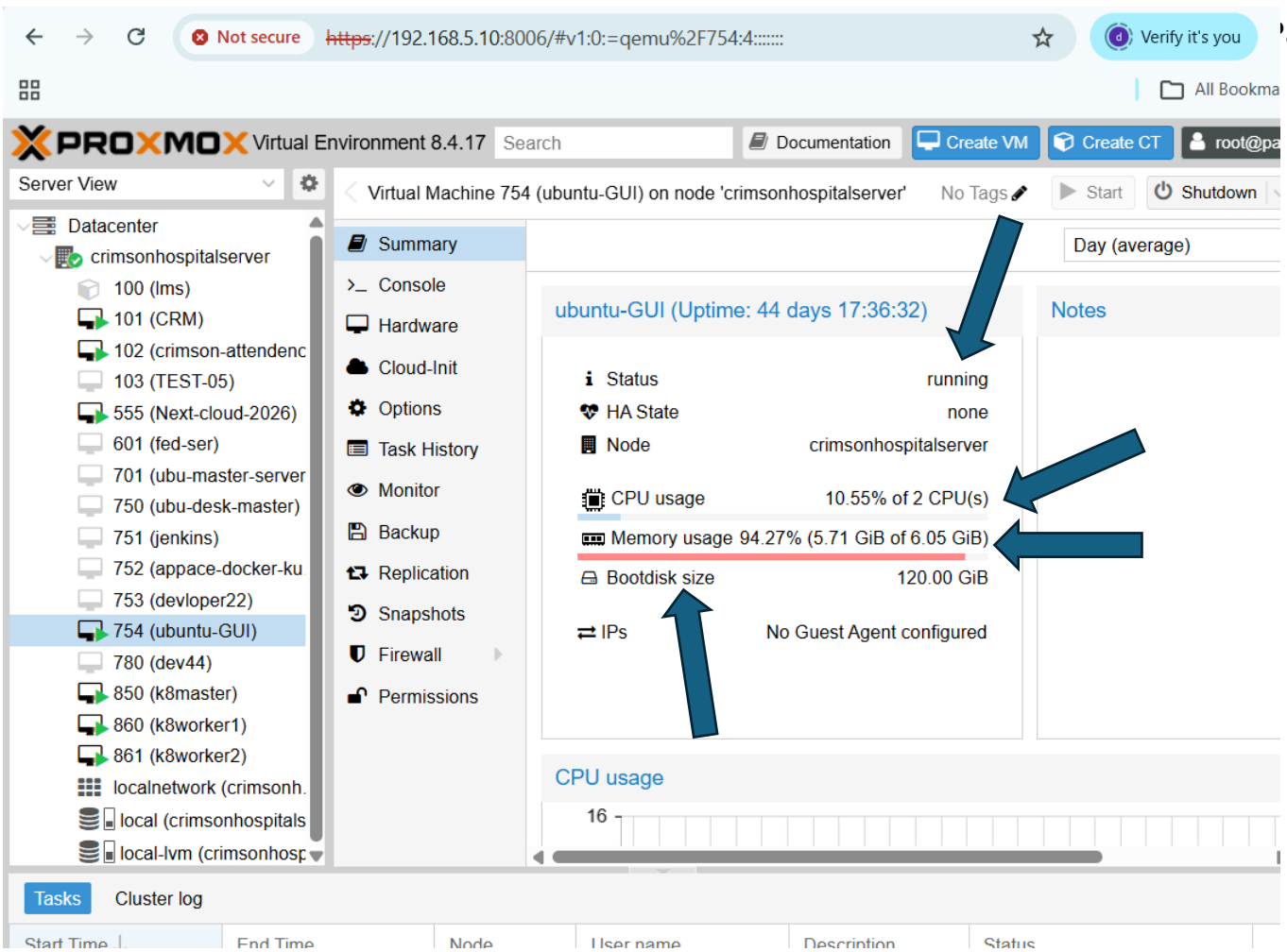
Step 10: Delete

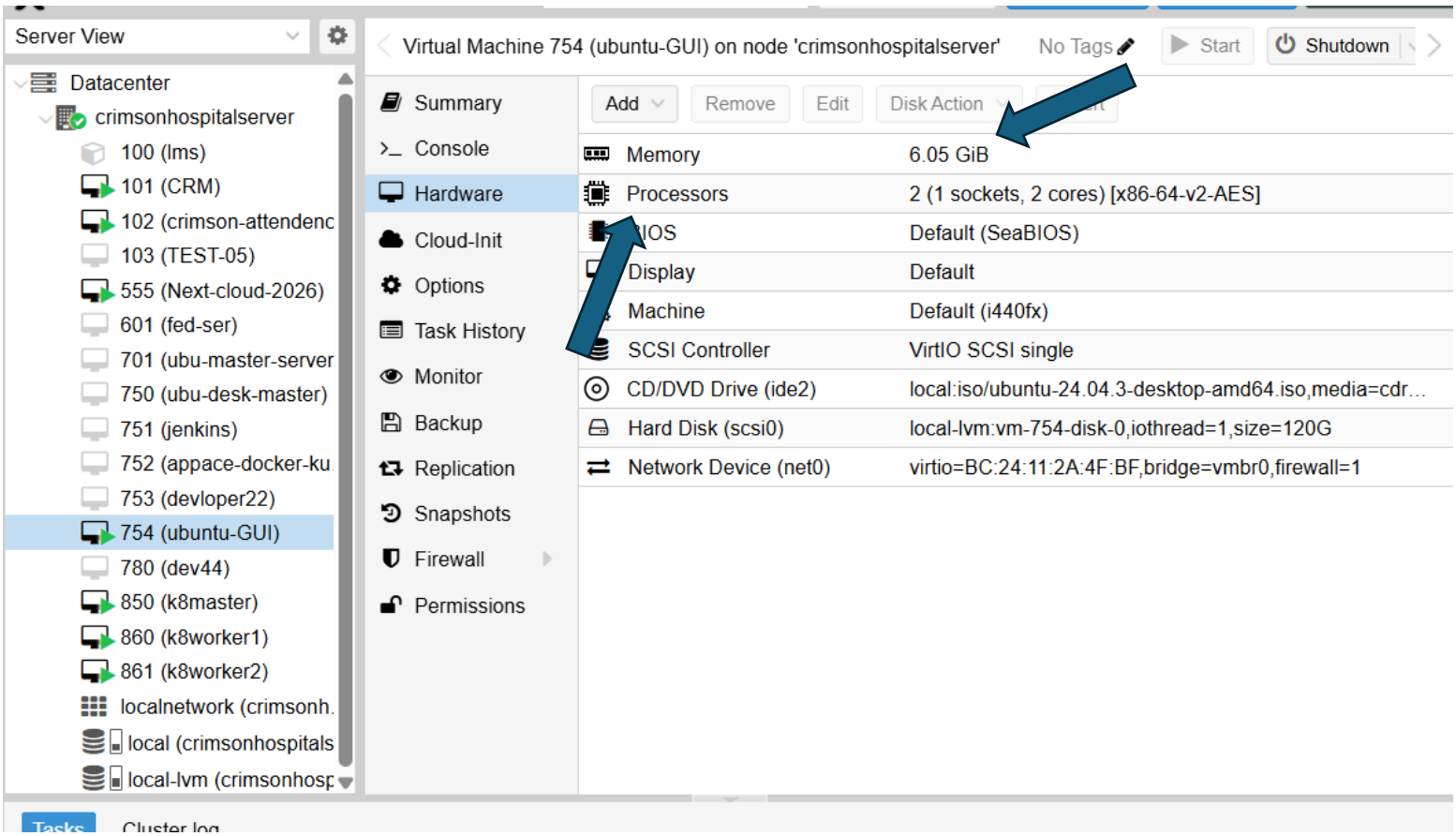
Remove VM when no longer needed.

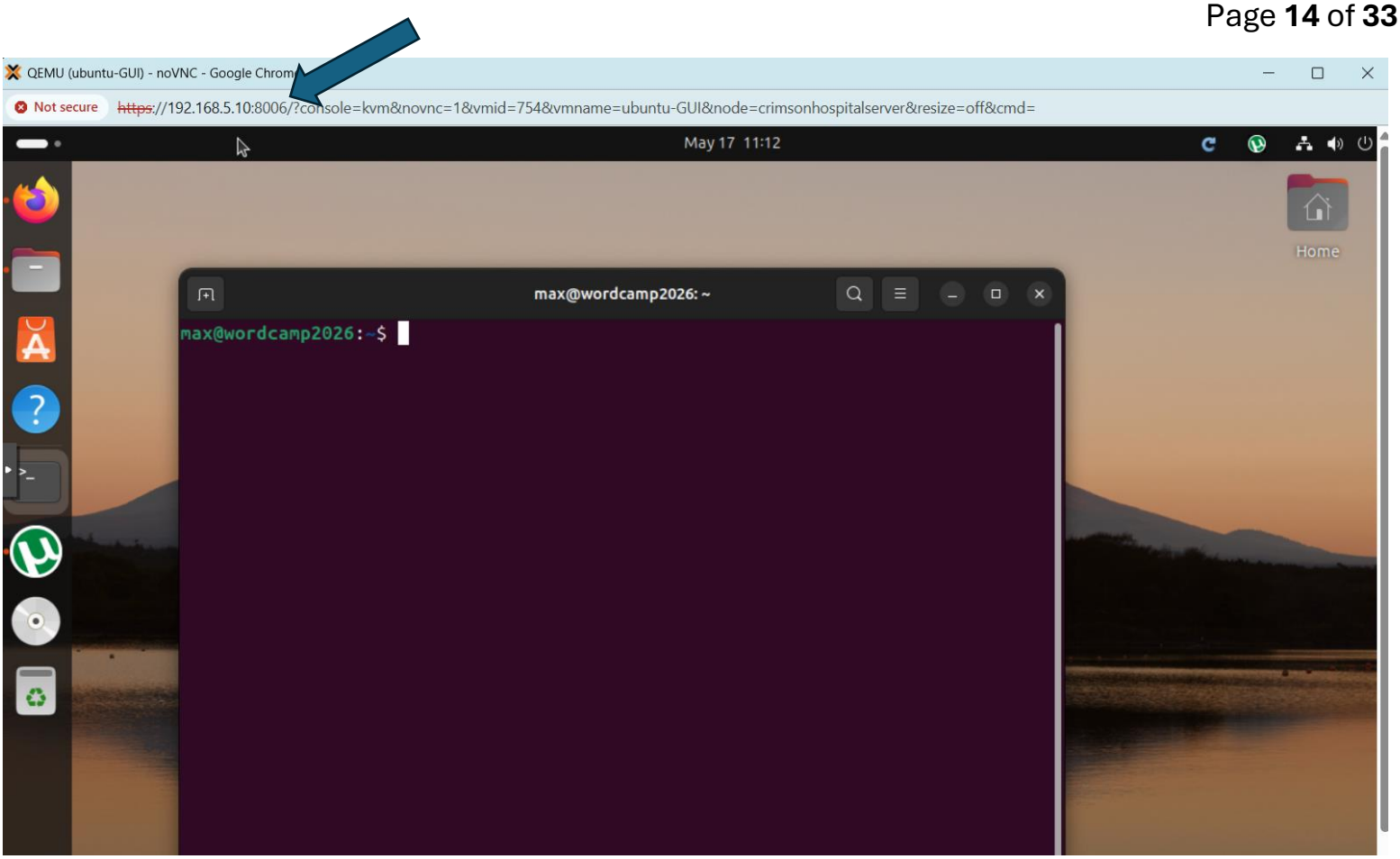
Proxmox Screenshot

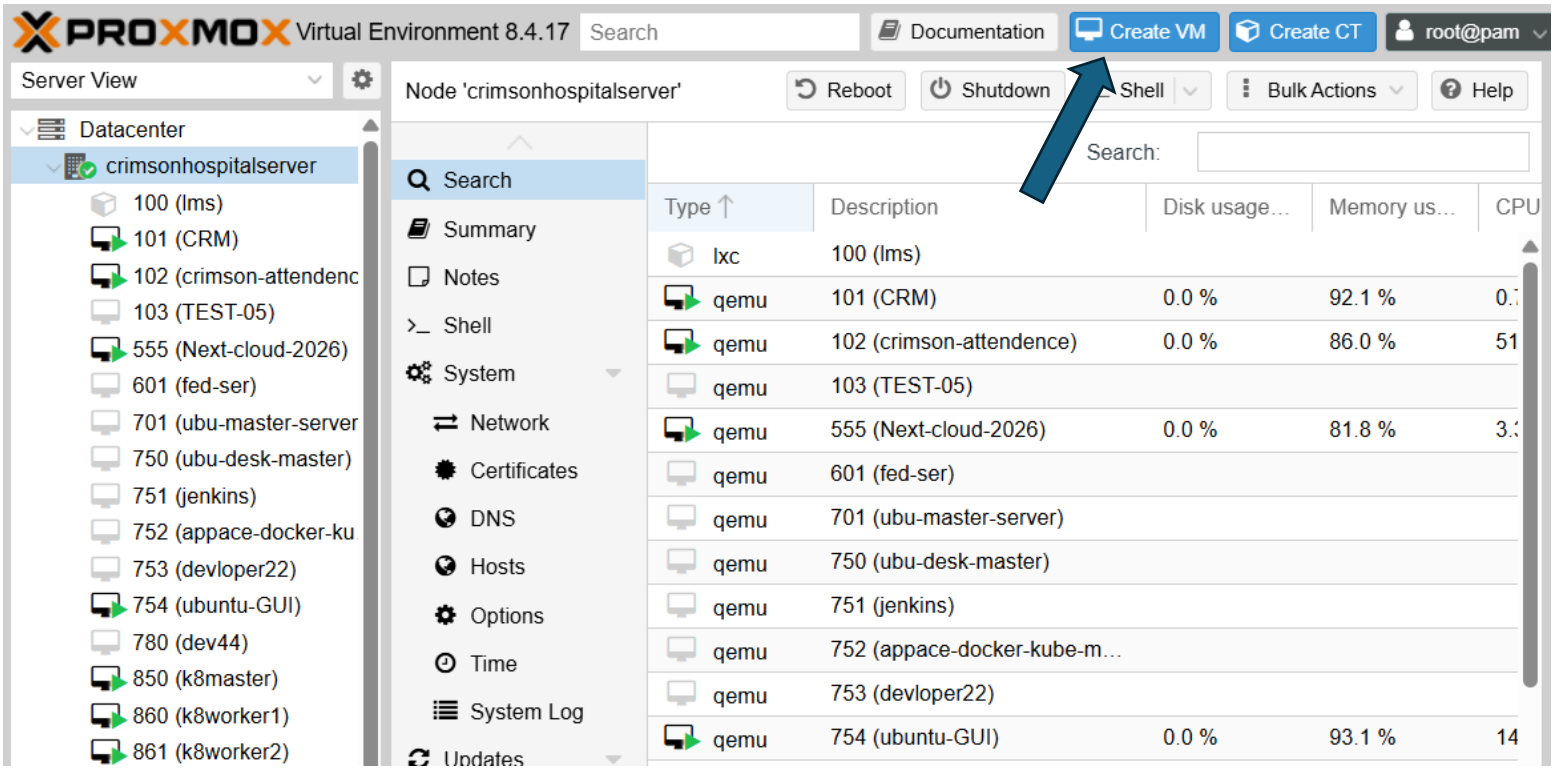


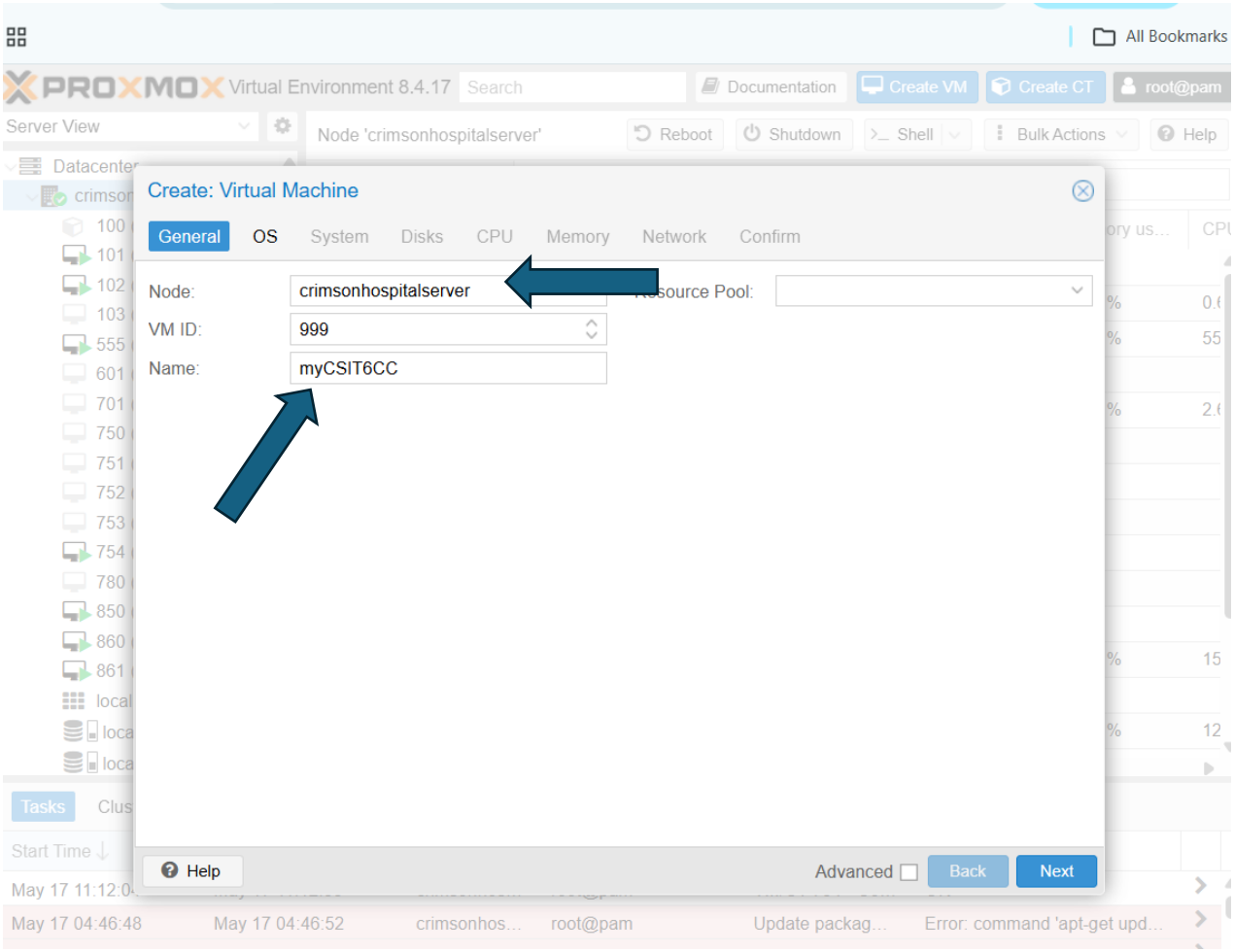


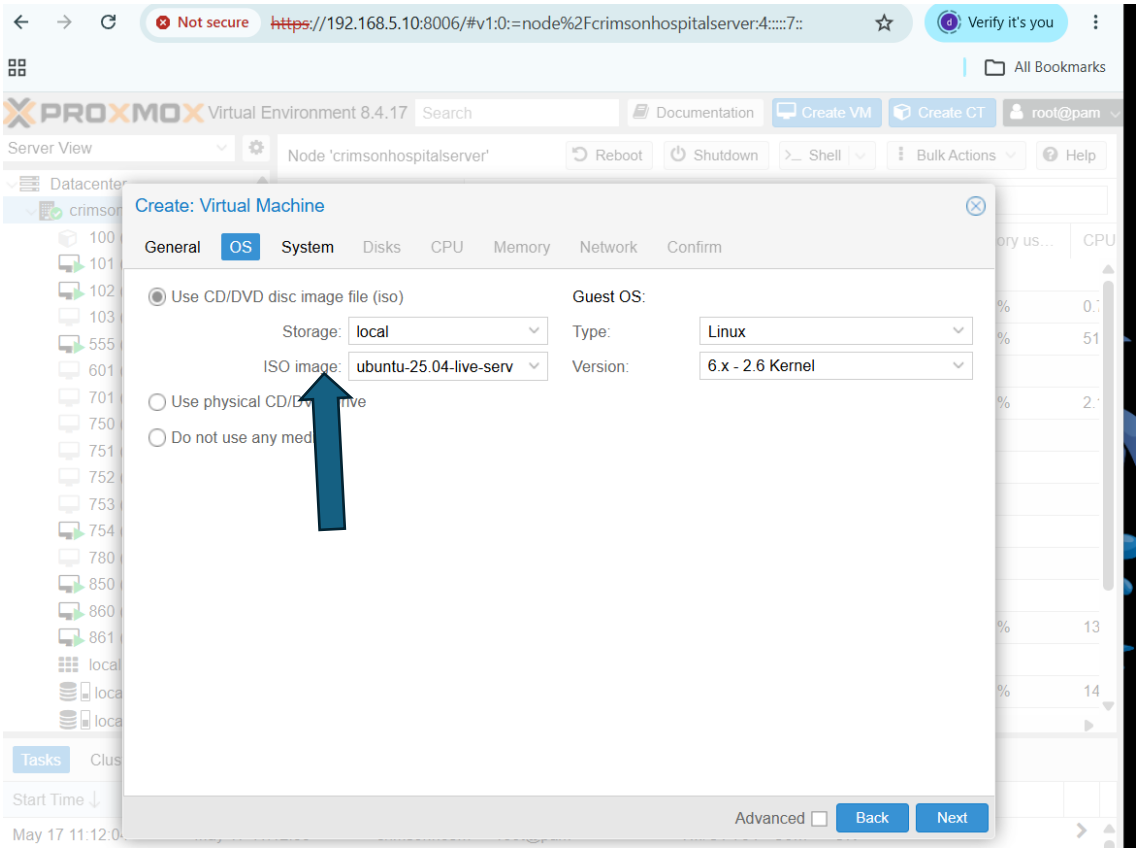

Not secure https://192.168.5.10:8006/#v1:0:=qemu%2F754:4::::::
Verify it's you
PROXMOX Virtual Environment 8.4.17
Search
Documentation
Create VM
Create CT
Server View
Datacenter
crimsonhospitalserver
100 (lms)
101 (CRM)
102 (crimson-attendenc
103 (TEST-05)
555 (Next-cloud-2026)
601 (fed-ser)
701 (ubu-master-server
750 (ubu-desk-master)
751 (jenkins)
752 (appace-docker-ku
753 (devloper22)
754 (ubuntu-GUI)
780 (dev44)
850 (k8master)
860 (k8worker1)
861 (k8worker2)
localnetwork (crimsonh
local (crimsonhospitals
local-lvm (crimsonhosp
Virtual Machine 754 (ubuntu-GUI) on node 'crimsonhospitalserver'
No Tags
Start
Shutdown
Summary
Console
Hardware
Cloud-Init
Options
Task History
Monitor
Backup
Replication
Snapshots
Firewall
Permissions
Day (average)
ubuntu-GUI (Uptime: 44 days 17:36:32)
Notes
Status running
HA State none
Node crimsonhospitalserver
CPU usage 10.55% of 2 CPU(s)
Memory usage 94.27% (5.71 GiB of 6.05 GiB)
Bootdisk size 120.00 GiB
IPs No Guest Agent configured
CPU usage
16
Tasks
Cluster log
Start Time
End Time
Node
User name
Description
Status

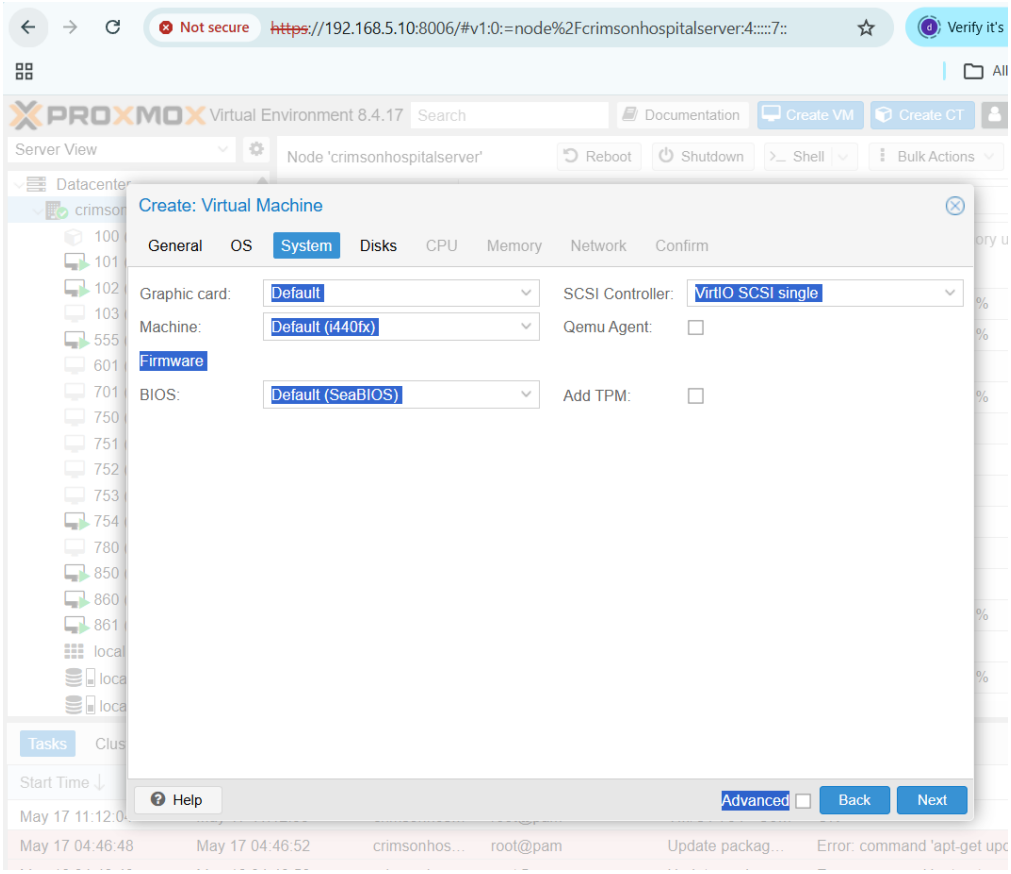

Server View
Datacenter
crimsonhospitalserver
100 (lms)
101 (CRM)
102 (crimson-attendenc
103 (TEST-05)
555 (Next-cloud-2026)
601 (fed-ser)
701 (ubu-master-server
750 (ubu-desk-master)
751 (jenkins)
752 (appace-docker-ku
753 (devloper22)
754 (ubuntu-GUI)
780 (dev44)
850 (k8master)
860 (k8worker1)
861 (k8worker2)
localnetwork (crimsonh
local (crimsonhospitals
local-lvm (crimsonhosp
Virtual Machine 754 (ubuntu-GUI) on node 'crimsonhospitalserver'
No Tags
Start
Shutdown
Summary
Console
Hardware
Cloud-Init
Options
Task History
Monitor
Backup
Replication
Snapshots
Firewall
Permissions
Add
Remove
Edit
Disk Action
Memory 6.05 GiB
Processors 2 (1 sockets, 2 cores) [x86-64-v2-AES]
BIOS Default (SeaBIOS)
Display Default
Machine Default (i440fx)
SCSI Controller VirtIO SCSI single
CD/DVD Drive (ide2) local:iso/ubuntu-24.04.3-desktop-amd64.iso,media=cdr...
Hard Disk (scsi0) local-lvm:vm-754-disk-0,iothread=1,size=120G
Network Device (net0) virtio=BC:24:11:2A:4F:BF,bridge=vmbr0,firewall=1
Tasks
Cluster log

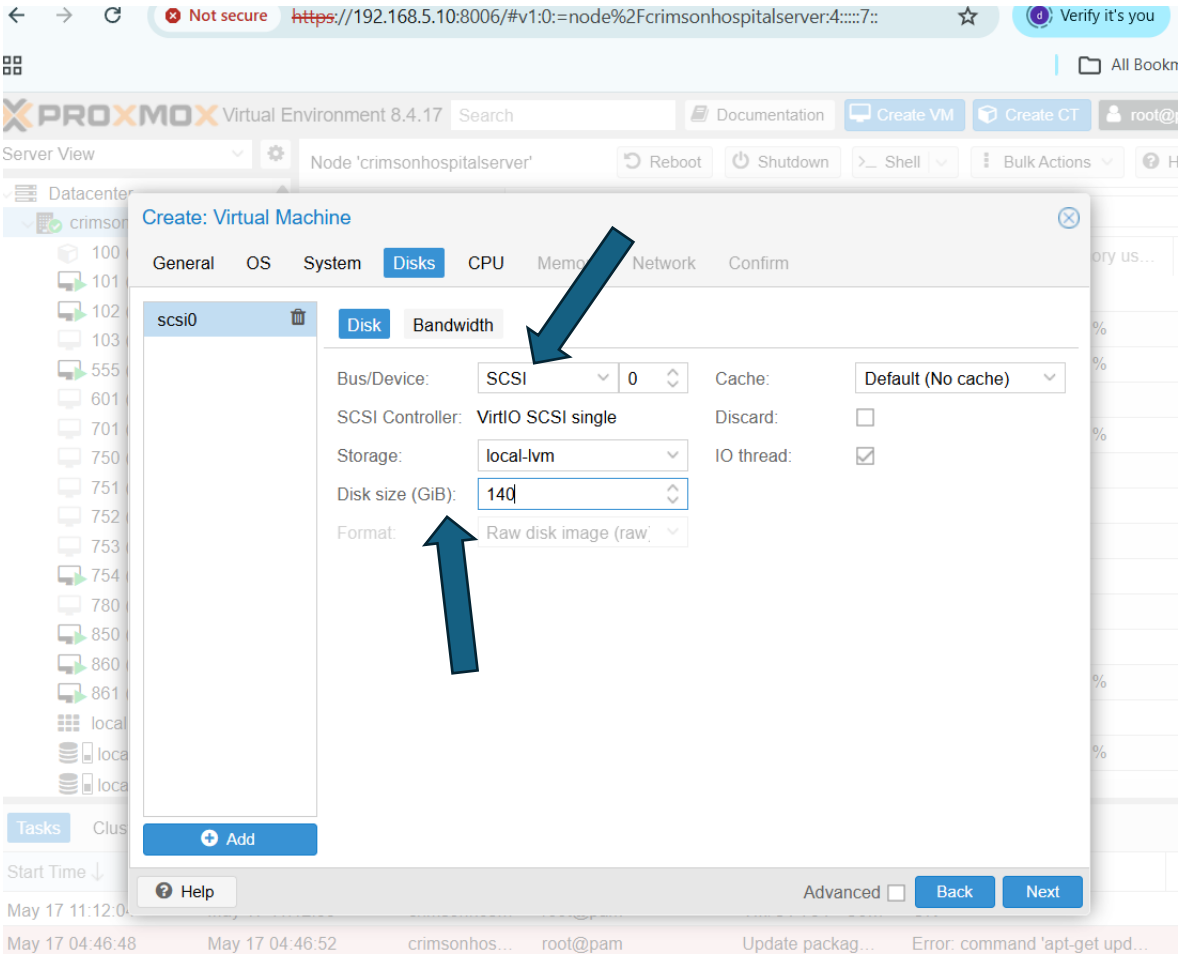

QEMU (ubuntu-GUI) - noVNC - Google Chrom
Not secure
https://192.168.5.10:8006/?console=kvm&novnc=1&vmid=754&vmname=ubuntu-GUI&node=crimsonhospitalserver&resize=off&cmd=
May 17 11:12
Home
max@wordcamp2026: ~
max@wordcamp2026:~$

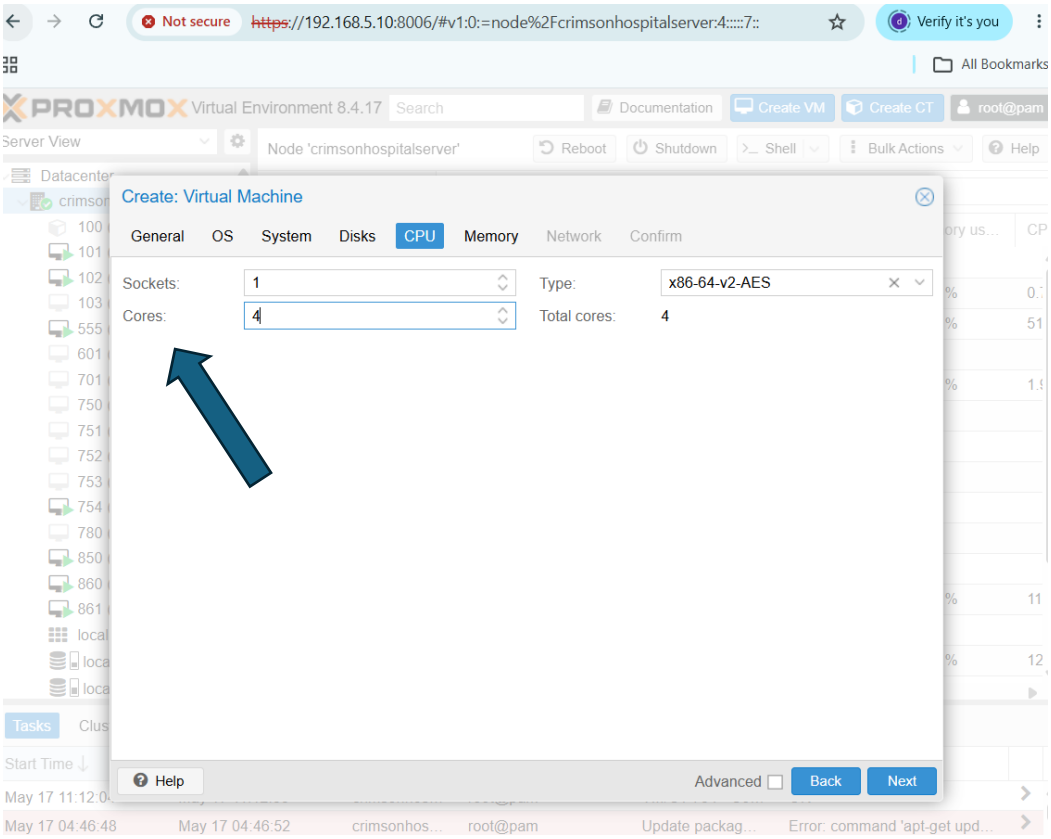

PROXMOX Virtual Environment 8.4.17
Search
Documentation
Create VM
Create CT
root@pam
Server View
Node 'crimsonhospitalserver'
Reboot
Shutdown
Shell
Bulk Actions
Help
Datacenter
crimsonhospitalserver
100 (lms)
101 (CRM)
102 (crimson-attendenc
103 (TEST-05)
555 (Next-cloud-2026)
601 (fed-ser)
701 (ubu-master-server
750 (ubu-desk-master)
751 (jenkins)
752 (appace-docker-ku
753 (devloper22)
754 (ubuntu-GUI)
780 (dev44)
850 (k8master)
860 (k8worker1)
861 (k8worker2)
Search
Summary
Notes
Shell
System
Network
Certificates
DNS
Hosts
Options
Time
System Log
Updates
Search:
Type
Description
Disk usage...
Memory us...
CPU
lxc 100 (lms)
qemu 101 (CRM) 0.0 % 92.1 %
qemu 102 (crimson-attendence) 0.0 % 86.0 %
qemu 103 (TEST-05)
qemu 555 (Next-cloud-2026) 0.0 % 81.8 %
qemu 601 (fed-ser)
qemu 701 (ubu-master-server)
qemu 750 (ubu-desk-master)
qemu 751 (jenkins)
qemu 752 (appace-docker-kube-m...
qemu 753 (devloper22)
qemu 754 (ubuntu-GUI) 0.0 % 93.1 %

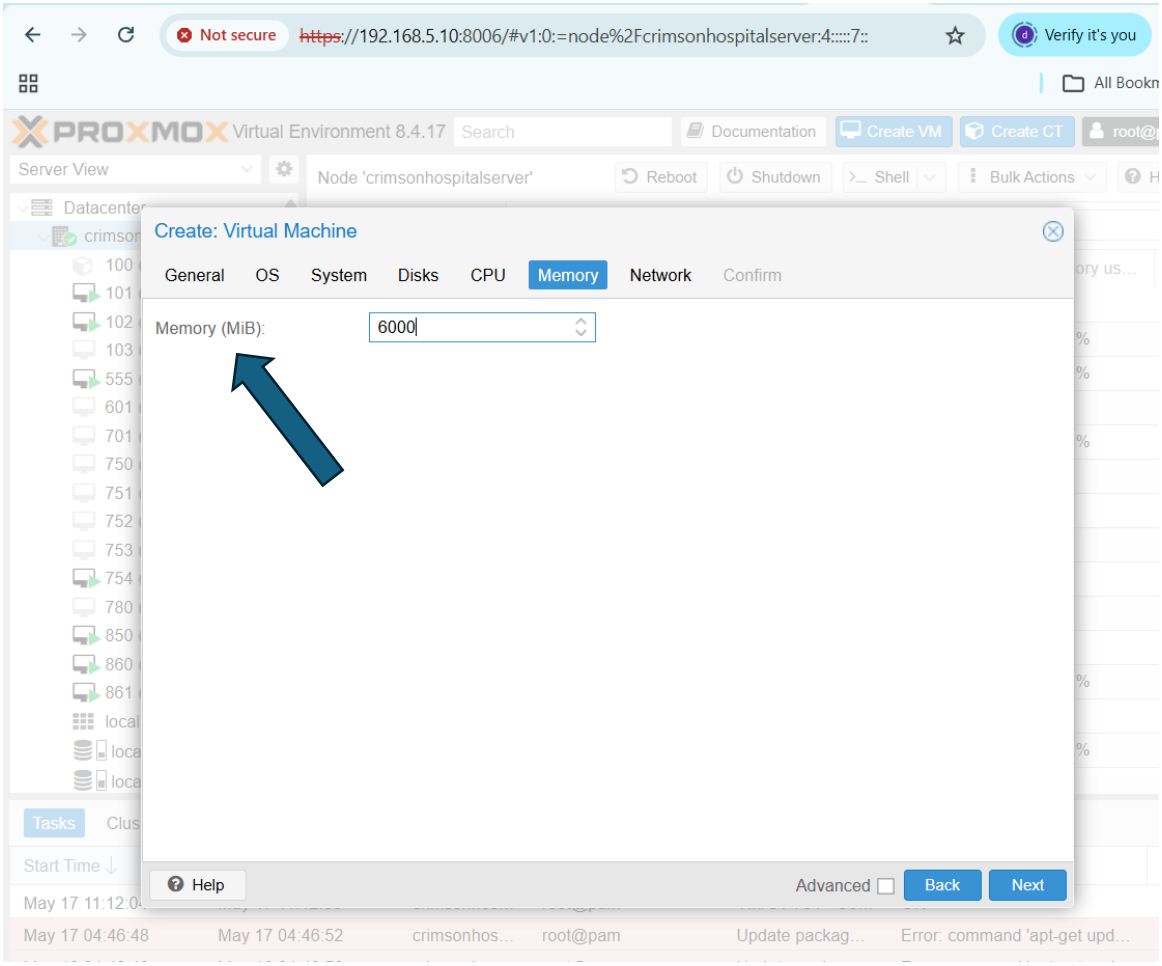

Create: Virtual Machine
General
OS
System
Disks
CPU
Memory
Network
Confirm
Node:
crimsonhospitalserver
Resource Pool:
VM ID:
999
Name:
myCSIT6CC
Help
Advanced
Back
Next

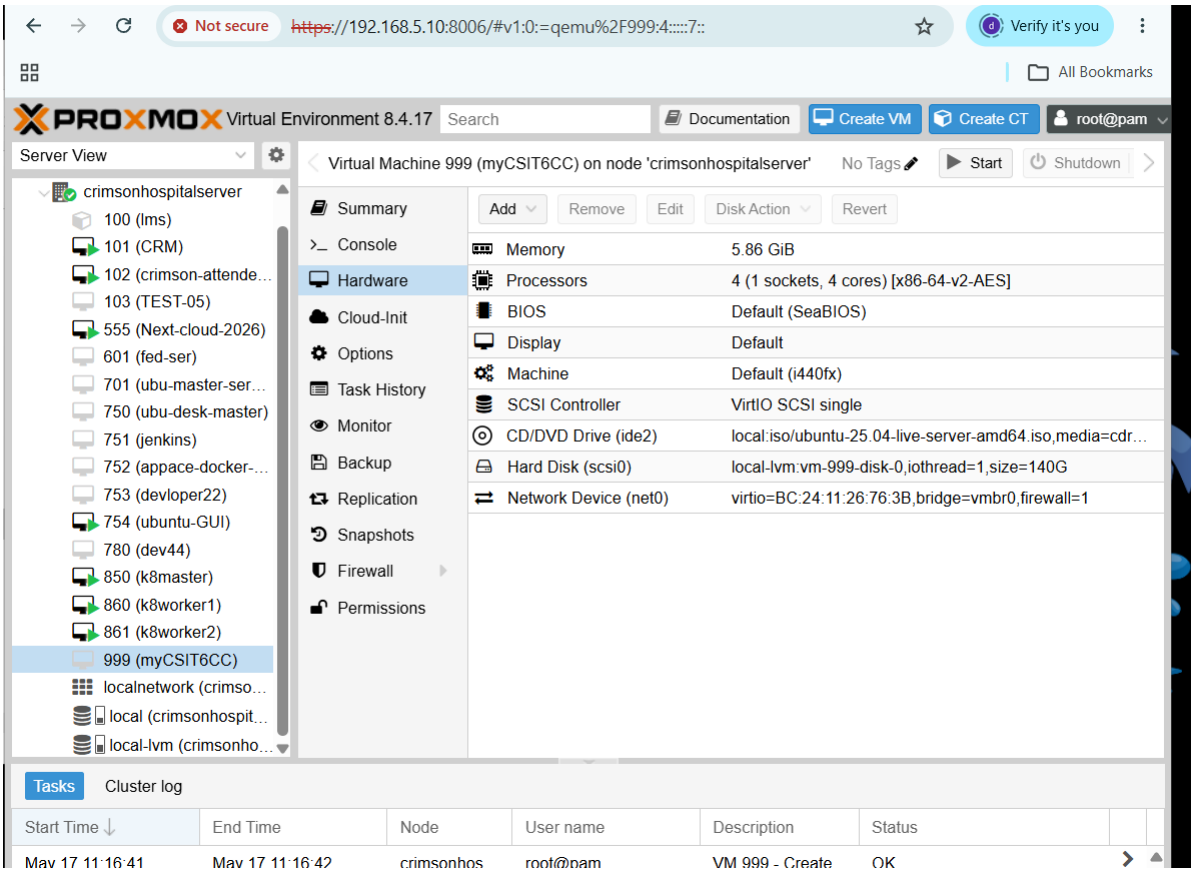

Create: Virtual Machine
General
OS
System
Disks
CPU
Memory
Network
Confirm
Use CD/DVD disc image file (iso)
Storage:
local
ISO image:
ubuntu-25.04-live-serv
Guest OS:
Type:
Linux
Version:
6.x - 2.6 Kernel
Advanced
Back
Next

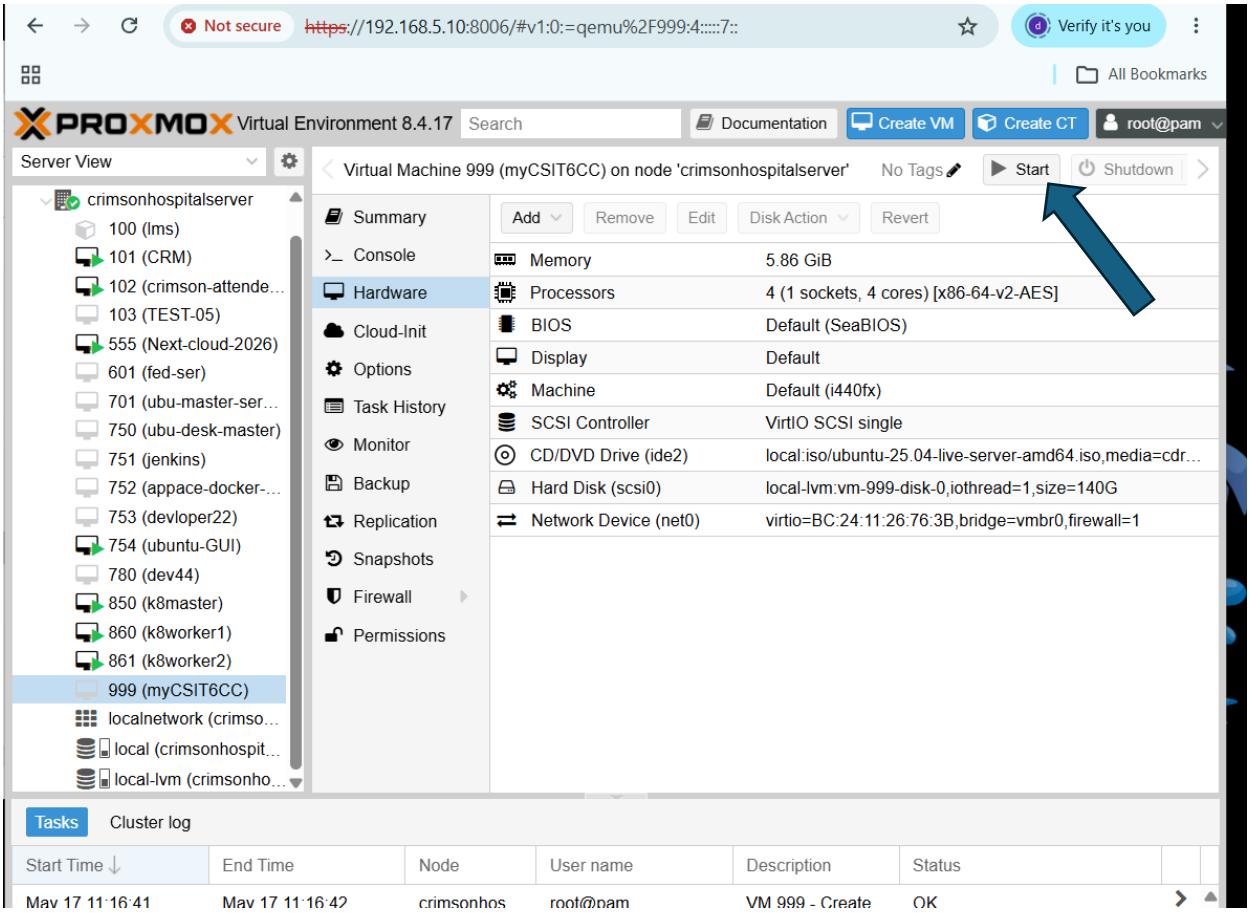

Not secure
https://192.168.5.10:8006/#v1:0:=node%2Fcrimsonhospitalserver:4:::::7::
Verify it's
PROXMOX Virtual Environment 8.4.17
Search
Documentation
Create VM
Create CT
Server View
Node 'crimsonhospitalserver'
Reboot
Shutdown
Shell
Bulk Actions
Create: Virtual Machine
General
OS
System
Disks
CPU
Memory
Network
Confirm
Graphic card:
Default
SCSI Controller:
VirtIO SCSI single
Machine:
Default (i440fx)
Qemu Agent:
Firmware
BIOS:
Default (SeaBIOS)
Add TPM:
Help
Advanced
Back
Next


Not secure
https://192.168.5.10:8006/#v1:0:=node%2Fcrimsonhospitalserver:4:::::7::
Verify it's you
PROXMOX Virtual Environment 8.4.17
Search
Documentation
Create VM
Create CT
Server View
Node 'crimsonhospitalserver'
Reboot
Shutdown
Shell
Bulk Actions
Create: Virtual Machine
General
OS
System
Disks
CPU
Network
Confirm
scsi0
Disk
Bandwidth
Bus/Device:
SCSI
0
Cache:
Default (No cache)
SCSI Controller:
VirtIO SCSI single
Discard:
Storage:
local-lvm
IO thread:
Disk size (GiB):
140
Format:
Raw disk image (raw)
Add
Help
Advanced
Back
Next


Not secure
https://192.168.5.10:8006/#v1:0:=node%2Fcrimsonhospitalserver:4:::::7::
Verify it's you
All Bookmarks
PROXMOX Virtual Environment 8.4.17
Search
Documentation
Create VM
Create CT
root@pam
Server View
Node 'crimsonhospitalserver'
Reboot
Shutdown
Shell
Bulk Actions
Help
Create: Virtual Machine
General
OS
System
Disks
CPU
Memory
Network
Confirm
Sockets:
1
Cores:
4
Type:
x86-64-v2-AES
Total cores:
4
Help
Advanced
Back
Next
Tasks
Start Time
May 17 04:46:48
May 17 04:46:52
crimsonhos...
root@pam
Update packag...
Error: command 'apt-get upd...


Not secure
https://192.168.5.10:8006/#v1:0:=node%2Fcrimsonhospitalserver:4:::::7::
Verify it's you
PROXMOX Virtual Environment 8.4.17
Search
Documentation
Create VM
Create CT
Server View
Node 'crimsonhospitalserver'
Reboot
Shutdown
Shell
Bulk Actions
Create: Virtual Machine
General
OS
System
Disks
CPU
Memory
Network
Confirm
Memory (MiB):
6000
Help
Advanced
Back
Next
Tasks
Start Time
May 17 04:46:48
May 17 04:46:52
crimsonhos...
root@pam
Update packag...
Error: command 'apt-get upd...


Not secure
https://192.168.5.10:8006/#v1:0:=qemu%2F999:4::::7::
Verify it's you
All Bookmarks
PROXMOX Virtual Environment 8.4.17
Search
Documentation
Create VM
Create CT
root@pam
Server View
crimsonhospitalserver
100 (lms)
101 (CRM)
102 (crimson-attende...
103 (TEST-05)
555 (Next-cloud-2026)
601 (fed-ser)
701 (ubu-master-ser...
750 (ubu-desk-master)
751 (jenkins)
752 (appace-docker-...
753 (devloper22)
754 (ubuntu-GUI)
780 (dev44)
850 (k8master)
860 (k8worker1)
861 (k8worker2)
999 (myCSIT6CC)
localnetwork (crimso...
local (crimsonhospit...
local-lvm (crimsonho...
Virtual Machine 999 (myCSIT6CC) on node 'crimsonhospitalserver'
No Tags
Start
Shutdown
Summary
Console
Hardware
Cloud-Init
Options
Task History
Monitor
Backup
Replication
Snapshots
Firewall
Permissions
Add
Remove
Edit
Disk Action
Revert
Memory 5.86 GiB
Processors 4 (1 sockets, 4 cores) [x86-64-v2-AES]
BIOS Default (SeaBIOS)
Display Default
Machine Default (i440fx)
SCSI Controller VirtIO SCSI single
CD/DVD Drive (ide2) local:iso/ubuntu-25.04-live-server-amd64.iso,media=cdr...
Hard Disk (scsi0) local-lvm:vm-999-disk-0,iothread=1,size=140G
Network Device (net0) virtio=BC:24:11:26:76:3B,bridge=vmbr0,firewall=1
Tasks
Cluster log
Start Time
End Time
Node
User name
Description
Status
May 17 11:16:41
May 17 11:16:42
crimsonhos
root@pam
VM 999 - Create
OK


Not secure
https://192.168.5.10:8006/#v1:0:=qemu%2F999:4::::7::
Verify it's you
All Bookmarks
PROXMOX Virtual Environment 8.4.17
Search
Documentation
Create VM
Create CT
root@pam
Server View
crimsonhospitalserver
100 (lms)
101 (CRM)
102 (crimson-attende...
103 (TEST-05)
555 (Next-cloud-2026)
601 (fed-ser)
701 (ubu-master-ser...
750 (ubu-desk-master)
751 (jenkins)
752 (appace-docker-...
753 (devloper22)
754 (ubuntu-GUI)
780 (dev44)
850 (k8master)
860 (k8worker1)
861 (k8worker2)
999 (myCSIT6CC)
localnetwork (crimso...
local (crimsonhospit...
local-lvm (crimsonho...
Virtual Machine 999 (myCSIT6CC) on node 'crimsonhospitalserver'
No Tags
Start
Shutdown
Summary
Console
Hardware
Cloud-Init
Options
Task History
Monitor
Backup
Replication
Snapshots
Firewall
Permissions
Add
Remove
Edit
Disk Action
Revert
Memory 5.86 GiB
Processors 4 (1 sockets, 4 cores) [x86-64-v2-AES]
BIOS Default (SeaBIOS)
Display Default
Machine Default (i440fx)
SCSI Controller VirtIO SCSI single
CD/DVD Drive (ide2) local:iso/ubuntu-25.04-live-server-amd64.iso,media=cdr...
Hard Disk (scsi0) local-lvm:vm-999-disk-0,iothread=1,size=140G
Network Device (net0) virtio=BC:24:11:26:76:3B,bridge=vmbr0,firewall=1
Tasks
Cluster log
Start Time
End Time
Node
User name
Description
Status
May 17 11:16:41
May 17 11:16:42
crimsonhos
root@pam
VM 999 - Create
OK

. 5.6 Introduction to containerization

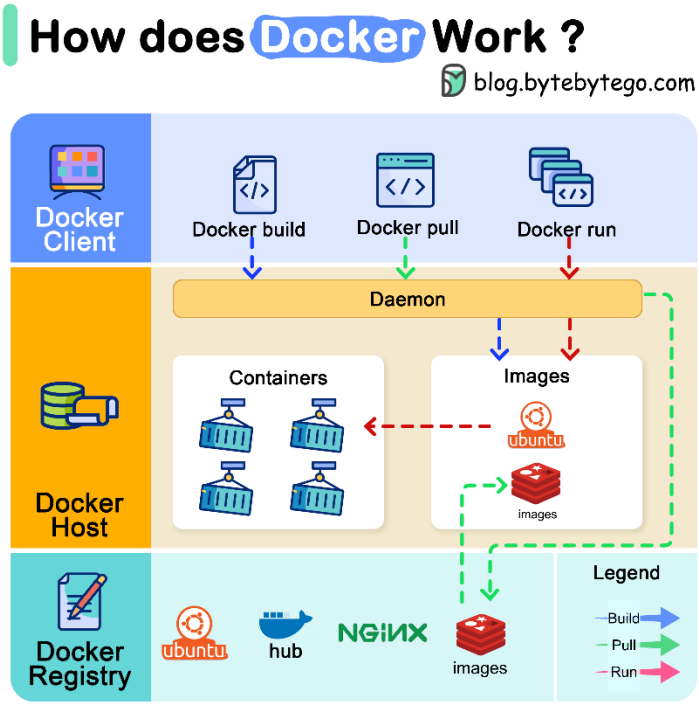
Meaning of Containerization

Containerization is a technology that packages an application and its required dependencies together so that it can run consistently in different environments.

A container includes:

. *Application code*
. *Libraries*
. *Dependencies*
. *Runtime environment*
. *Configuration files*

But unlike a VM, a container does not include a full guest operating system.



Simple Example

Suppose a Python web application needs:

. *Python 3.11*
. *Flask*
. *Pandas*
. *Nginx*
. *Specific environment variables*

Instead of installing these manually on every computer, we package them into a container.

Then the same container can run on:

. *Developer laptop*
. *Testing server*
. *Production server*
. *Cloud platform*

Containerization Architecture

```
+-----------------------------------+
|          Container 1              |
|      App + Libraries              |
+-----------------------------------+
|          Container 2              |
|      App + Libraries              |
+-----------------------------------+
|          Container 3              |
|      App + Libraries              |
+-----------------------------------+
|          Container Runtime        |
|       Docker / containerd         |
+-----------------------------------+
|          Host Operating System    |
+-----------------------------------+
|          Physical / Virtual Server |
+-----------------------------------+
```

Popular Container Tools

. Docker
. Podman
. containerd

Docker Practical Example

Goal

Run a simple web server using Docker.

A. Docker Installation on Ubuntu

sudo apt update
sudo apt install docker.io -y
sudo systemctl start docker
sudo systemctl enable docker
docker --version

B. Run Hello World Container

sudo docker run hello-world

This checks whether Docker is installed correctly.

C. Run Nginx Web Server Container

sudo docker run -d -p 8080:80 --name my-nginx nginx

Now open in browser:

http://localhost:8080

Or from another computer:

http://SERVER-IP:8080

D. Check Running Containers

sudo docker ps

E. Stop Container

sudo docker stop my-nginx

F. Start Container Again

sudo docker start my-nginx

G. Remove Container

sudo docker rm my-nginx

Docker Practical: Create Own Web Page

Step 1: Create Project Folder

mkdir cloud-demo
cd cloud-demo

Step 2: Create HTML File

nano index.html

Paste:

```
<!DOCTYPE html>
<html>
<head>
   <title>Cloud Computing Demo</title>
</head>
<body>
   <h1>Hello World! Welcome to Cloud Computing</h1>
   <p>This page is running inside a Docker container.</p>
</body>
</html>
```

Save the file.

Step 3: Create Dockerfile

nano Dockerfile

Paste:

```
FROM nginx:latest
COPY index.html /usr/share/nginx/html/index.html
```

Step 4: Build Docker Image

sudo docker build -t cloud-demo-web .

Step 5: Run Container

sudo docker run -d -p 8081:80 --name cloud-web cloud-demo-web

Open:

http://localhost:8081

Step 6: View Logs

sudo docker logs cloud-web

Step 7: Remove

sudo docker stop cloud-web
sudo docker rm cloud-web

Windows Docker Basic Commands

After installing Docker Desktop on Windows, open PowerShell:

docker --version
docker run hello-world
docker run -d -p 8080:80 --name win-nginx nginx
docker ps
docker stop win-nginx
docker rm win-nginx

Open browser:

http://localhost:8080

. 5.7 Virtual Machine vs Container

Basic Difference

A virtual machine virtualizes hardware.
A container virtualizes the operating system environment.

VM vs Container Table

Feature	Virtual Machine	Container
Virtualizes	Hardware	Operating system
OS	Each VM has full OS	Shares host OS kernel
Size	Large	Small
Startup time	Slow	Fast
Performance	More overhead	Lightweight
Isolation	Strong	Process-level isolation
Portability	Medium	High
Boot time	Minutes	Seconds
Example tools	VMware, VirtualBox, KVM	Docker, Podman, Kubernetes
Best for	Full OS environments	Microservices and apps

Data-Based Comparison

Assume one physical server has:

Resource	Capacity
CPU	16 cores
RAM	64 GB
Storage	1 TB

Scenario A: Virtual Machines

Each VM uses:

CPU = 2 cores
RAM = 8 GB
Storage = 80 GB

Maximum VMs based on RAM:

64 GB ÷ 8 GB = 8 VMs

Maximum VMs based on CPU:

16 cores ÷ 2 cores = 8 VMs

Maximum VMs based on storage:

1000 GB ÷ 80 GB = 12 VMs approximately

So practical maximum:

8 VMs

Scenario B: Containers

Each container uses:

CPU = 0.5 core
RAM = 512 MB
Storage = 2 GB

RAM calculation:

64 GB = 65536 MB
65536 MB ÷ 512 MB = 128 containers

CPU calculation:

16 cores ÷ 0.5 core = 32 containers

Storage calculation:

1000 GB ÷ 2 GB = 500 containers

So practical maximum based on CPU:

32 containers

Interpretation

On the same server:

Technology	Approximate Capacity
Virtual Machine	8 VMs
Container	32 containers

Containers are more lightweight than VMs.

https://github.com/sanjeevlcc/notes_2081/blob/main/Artificial%20intelligence/Cloud%20Computing_2026/Notes_Blue/Unit%201/Lab5_Unit5_Docker_on_Power%20.txt

. 5.8 Introduction to serverless computing

Meaning of Serverless Computing

Serverless computing is a cloud computing model where developers write and deploy code without managing servers.

The cloud provider automatically handles:

. Server provisioning
. Scaling
. Runtime
. Availability
. Infrastructure management

The term serverless does not mean there is no server. It means the user does not manage the server.

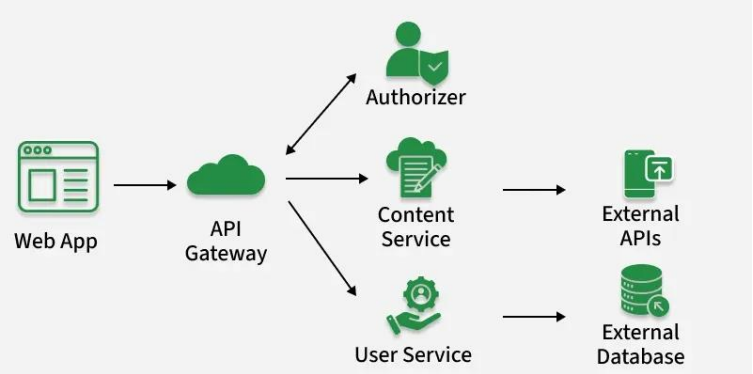
Simple Example

A student uploads a photo to a website.

Automatically:

. A function is triggered
. The image is resized
. A thumbnail is created
. The resized image is stored

The developer only writes the function logic. The cloud provider manages the infrastructure.



Serverless Platforms

. *AWS Lambda*
. *Azure Functions*
. *Google Cloud Functions*
. *Cloudflare Workers*
. *Vercel Functions*
. *Netlify Functions*

Serverless Architecture

User Action / Event
↓
Cloud Event Trigger
↓
Serverless Function
↓
Processing
↓
Output / Response

Benefits of Serverless

. *No server management*
. *Automatic scaling*
. *Pay only when function runs*
. *Fast development*
. *Good for event-based tasks*

Challenges of Serverless

. *Cold start delay*
. *Limited execution time*
. *Vendor lock-in*
. *Debugging can be difficult*
. *Not suitable for all applications*

Simple Serverless Example Concept

Use Case: Online Admission Form

When a student submits a form:

Event: Form submitted
Function:
. *Validate data*
. *Save to database*
. *Send confirmation email*
. *Notify admin*

The function runs only when the event occurs.

. 5.9 Event-driven functions

Meaning

Event-driven functions are functions that run automatically when a specific event occurs.

They are commonly used in serverless computing.

What is an Event?

An event is an action or change in the system.

Examples:

. *File uploaded*
. *Form submitted*

. *Payment completed*
. *Database record added*
. *Sensor sends data*
. *User logs in*
. *Message received*

Event-Driven Function Flow

Event Occurs
↓
Trigger Activates
↓
Function Executes
↓
Result is Generated

Examples of Event-Driven Functions

Event	Function Action
Image uploaded	Resize image
Payment completed	Send receipt
Student submits form	Send confirmation email
New user registers	Create user profile
CPU usage high	Send alert
IoT sensor sends temperature	Store reading in database

Data-Based Example: Temperature Alert

A cloud function receives sensor data from a server room.

Time	Temperature
10:00 AM	24°C
10:05 AM	26°C
10:10 AM	30°C
10:15 AM	35°C
10:20 AM	38°C

Condition:

If temperature > 35°C, send alert.

At 10:20 AM, temperature is 38°C.

Result:

Alert: Server room temperature is high.

Python Example: Event-Driven Function Logic

```
def temperature_alert(event):
    temperature = event["temperature"]

    if temperature > 35:
        return "Alert: Temperature is high!"
    else:
        return "Temperature is normal."

event_data = {
    "temperature": 38
}

result = temperature_alert(event_data)
print(result)
```

Output:

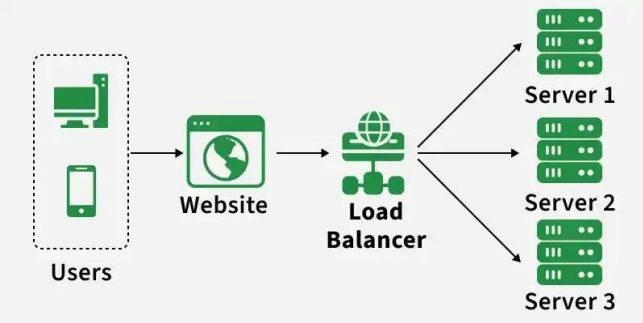
https://github.com/sanjeevlcc/notes_2081/blob/main/Artificial%20intelligence/Cloud%20Computing_2026/Notes_Blue/Unit%201/Lab6_Unit5_Serverless%20and%20eventFunctions%20.txt

. 5.10 Load balancing concepts

Meaning of Load Balancing

Load balancing is the process of distributing incoming network traffic across multiple servers.

It prevents one server from becoming overloaded.



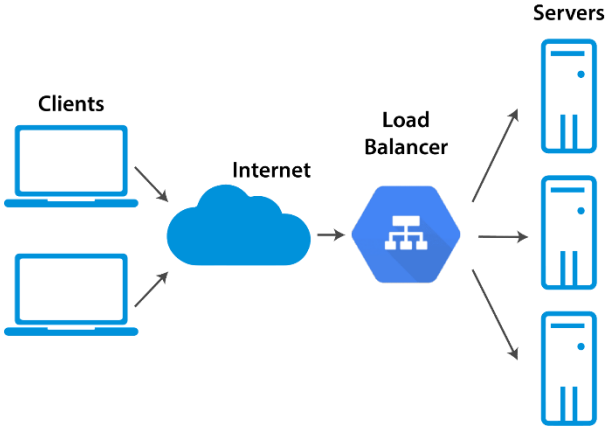
Simple Example

A website receives 9,000 user requests.

There are 3 servers.

Without load balancing:

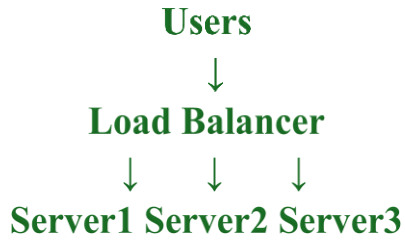
Server 1 may get too many requests.
Server 2 and Server 3 may remain underused.



With load balancing:

Server	Requests
Server 1	3,000
Server 2	3,000
Server 3	3,000

Load Balancing Diagram



Benefits of Load Balancing

. *Improves performance*
. *Prevents server overload*
. *Increases availability*
. *Supports scalability*
. *Improves user experience*

Common Load Balancing Algorithms

1. Round Robin

Requests are sent one by one to each server in order.

Example:

Request 1 → Server 1
Request 2 → Server 2
Request 3 → Server 3
Request 4 → Server 1

---

2. Least Connections

Request is sent to the server with the fewest active connections.

Example:

Server	Active Connections
Server 1	30
Server 2	10
Server 3	25

New request goes to:

Server 2

---

3. IP Hash

The user's IP address decides which server will handle the request.

Useful when the same user should connect to the same server.

---

4. Weighted Load Balancing

More powerful servers receive more traffic.

Example:

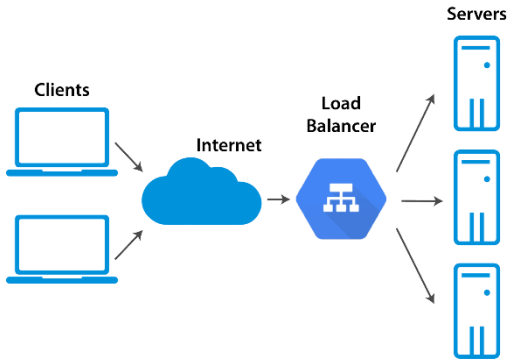
Server	Capacity	Weight
Server 1	High	5
Server 2	Medium	3
Server 3	Low	1

Server 1 receives more requests than Server 3.

Practical Nginx Load Balancing Example

Assume we have three application servers:

Server 1: 192.168.1.101:8080
Server 2: 192.168.1.102:8080
Server 3: 192.168.1.103:8080



Nginx configuration:

```
http {
   upstream backend_servers {
      server 192.168.1.101:8080;
      server 192.168.1.102:8080;
      server 192.168.1.103:8080;
   }
```

```
    server {
        listen 80;

        location / {
            proxy_pass http://backend_servers;
        }
    }
}
```

Docker Compose Load Balancing Demonstration

Goal

Run multiple web containers and balance traffic using Nginx.

Folder Structure

```
load-balance-demo/
│
├── docker-compose.yml
└── nginx.conf
```

---

docker-compose.yml

```
version: "3.8"

services:
  web1:
    image: nginx
    container_name: web1
    volumes:
      - ./web1.html:/usr/share/nginx/html/index.html

  web2:
    image: nginx
    container_name: web2
    volumes:
      - ./web2.html:/usr/share/nginx/html/index.html

  web3:
    image: nginx
    container_name: web3
    volumes:
      - ./web3.html:/usr/share/nginx/html/index.html

  loadbalancer:
    image: nginx
    container_name: loadbalancer
    ports:
      - "8080:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf
    depends_on:
      - web1
      - web2
      - web3
```

---

nginx.conf

```
events {}

http {
    upstream web_servers {
        server web1:80;
        server web2:80;
        server web3:80;
    }

    server {
        listen 80;

        location / {
            proxy_pass http://web_servers;
        }
    }
}
```

Create HTML Files

```
echo "<h1>This is Web Server 1</h1>" > web1.html
echo "<h1>This is Web Server 2</h1>" > web2.html
echo "<h1>This is Web Server 3</h1>" > web3.html
```

Run

docker compose up -d

Open:

http://localhost:8080

Refresh multiple times to observe response from different backend servers.

Stop

docker compose down

. 5.11 Serverless vs containerized workloads

Meaning of Containerized Workload

A containerized workload is an application running inside containers.

Example:

. *Docker container*
. *Kubernetes pod*
. *Microservice application*

Meaning of Serverless Workload

A serverless workload is code that runs only when triggered by an event.

Example:

. *AWS Lambda function*
. *Azure Function*
. *Google Cloud Function*

Serverless vs Containerized Workloads

Feature	Serverless	Containerized
Server management	Managed by provider	User manages container environment
Runtime	Runs on event	Runs continuously
Scaling	Automatic	Manual or Kubernetes-based
Cost	Pay per execution	Pay for running resources
Startup	May have cold start	Usually always running
Control	Less control	More control
Best for	Short event-based tasks	Long-running applications

Feature	Serverless	Containerized
Examples	AWS Lambda	Docker, Kubernetes

When to Use Serverless

Use serverless when:

- *Task is event-based*
- *Function runs for short time*
- *Traffic is unpredictable*
- *You want minimal server management*
- *You need automatic scaling*

Examples

- *Sending email after form submission*
- *Resizing uploaded image*
- *Processing payment notification*
- *Running scheduled backup task*
- *Sending alert when CPU is high*

When to Use Containers

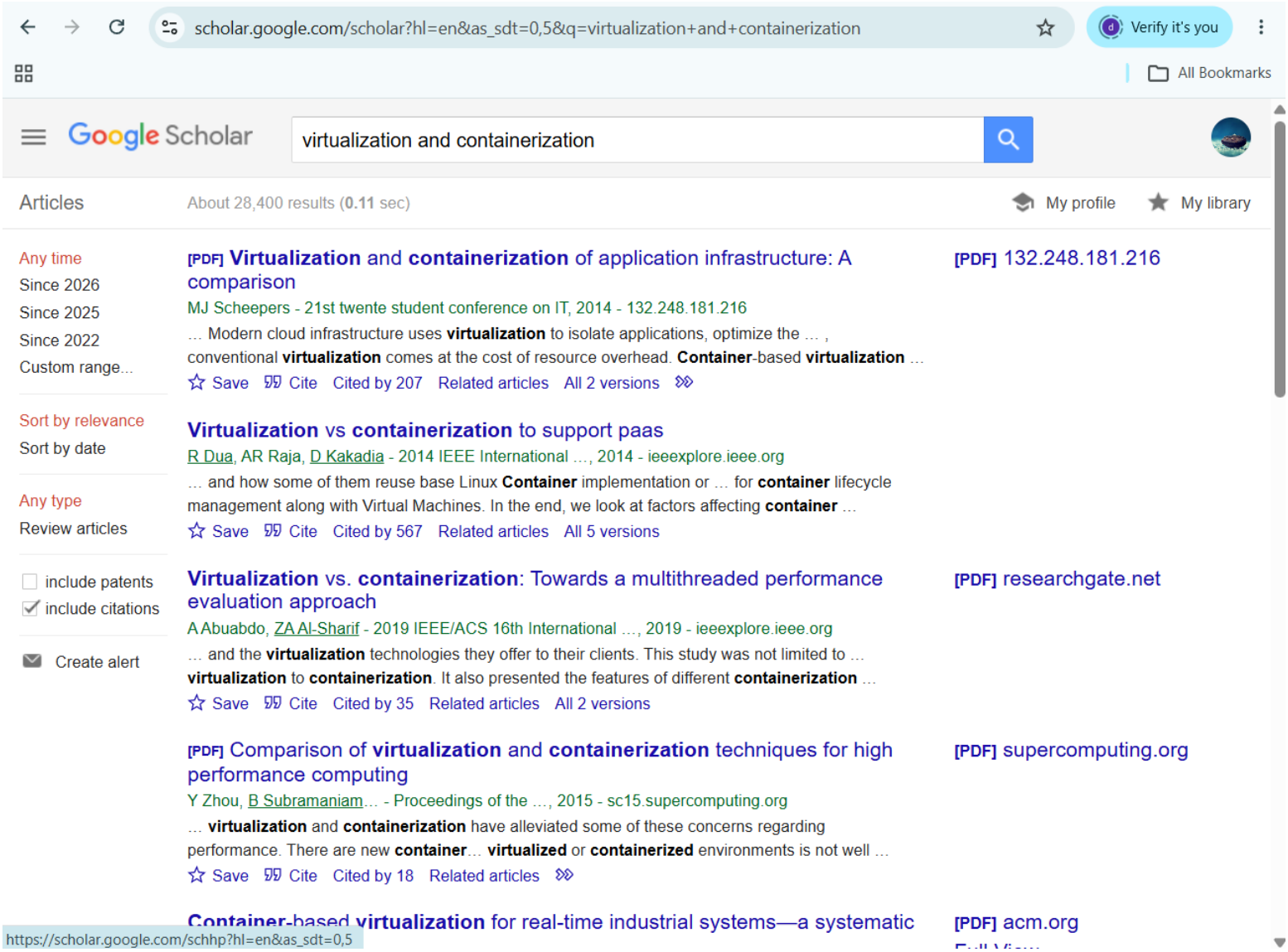
Use containers when:

- *Application runs continuously*
- *You need more control*
- *You have microservices*
- *You need custom runtime*
- *Application has complex dependencies*

Examples

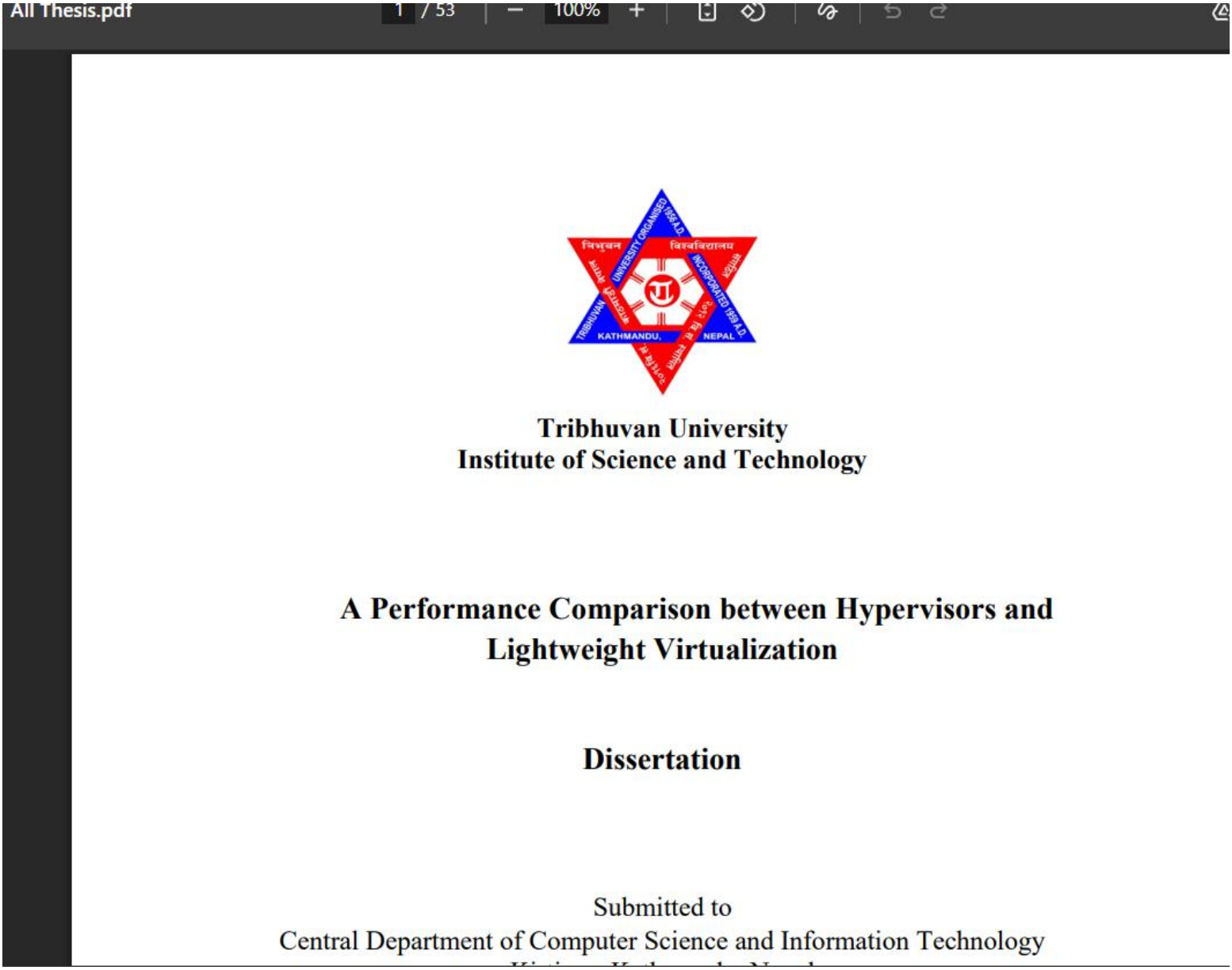
- *Web application*
- *API server*
- *Database service*
- *ERP system*
- *Learning Management System*
- *AI model serving application*

PAPER/ CASE



1. http://132.248.181.216/DC/MaquinasVirtuales/CursoMaquinasVirtuales/VirtualizacionEnLinuxCon-Containers/539ae779eb69a.pdf
2. https://sc15.supercomputing.org/sites/all/themes/SC15images/tech_poster/poster_files/post239s2-file3.pdf
3. https://dl.acm.org/doi/pdf/10.1145/3617591
4. https://elibrary.tucl.edu.np/JQ99OgQIizUxyjI9nB0on9OyLkqsGIf4/api/core/bitstreams/d8137104-fa40-48e7-883f-a951e03e1bb2/content

All Thesis.pdf 1 / 53 − 100% +



Tribhuvan University
Institute of Science and Technology

A Performance Comparison between Hypervisors and Lightweight Virtualization

Dissertation

Submitted to
Central Department of Computer Science and Information Technology

https://elibrary.tucl.edu.np/JQ99OgQIizUxyjI9nB0on9OyLkqsGIf4/api/core/bitstreams/d8137104-fa40-48e7-883f-a951e03e1bb2/content

Basic Questions

1. What is virtualization, and why is it considered a foundation technology for cloud computing?
2. What are the major differences between hypervisor-based virtualization and container-based lightweight virtualization?
3. How do KVM and LXC/LXD differ in terms of architecture, resource sharing, and guest operating system support?
4. What roles do namespaces and control groups play in Linux container-based virtualization?
5. Which benchmarking tools were used in the thesis to measure CPU, memory, disk I/O, and web server response time?

Analytical Questions

6. Why does lightweight virtualization generally produce lower overhead than hypervisor-based virtualization?

7. How does sharing the host kernel affect the performance and limitations of Linux containers?

8. Why might LXC/LXD outperform KVM in CPU, memory, disk I/O, and Apache web server response time?

9. How does the use of qcow2 disk image format in KVM affect disk I/O performance compared with LXC?

10. What does lower standard deviation indicate in the performance comparison between KVM and LXC/LXD?